



Tazama Connection Studio– FSD

FUNCTIONAL SPECIFICATIONS DOCUMENT

Version 0.3 (Final Review Draft)

14.07.2025

Document Information

Project	Tazama Development
Document Title	Tazama Connection Studio– FSD
Prepared by	Behjet Ansari – Senior BA @ Paysys Labs
Version Number	0.3
Version Date	14/07/2025

Document Revision History

Date	Author	Version	Change Reference
05/06/2025	Behjet Ansari	0.2	Baseline version shared with Tazama for review.
14/07/2025	Behjet Ansari	0.3	Updated version with finalized design.

Table of Contents

Table of Contents	2
Introduction	4
1.1 Purpose of the Document	4
1.2 Scope of the System	4
Overall Description	5
2.1 Operating Environment	5
System Architecture Overview	6
3.1 High-Level Architecture	6
3.2 Configuration Flow	8
3.3 Transactional Flow	9
4. Functional Overview	10
4.1 General Requirements	10
Decoupled Operation from DEMS Runtime	10
User Types	13
User Type Responsibilities	14
Data Enrichment Use Cases	16
4.2 API Endpoint Construction and Deployment	18
Auto-generation of API Endpoints	18
Deployment into DEMS and Data Enrichment Layers	26
Custom API Paths	26
Role-Based Access Control via KeyCloak	27
Server-Side Validation	27
Endpoint Versioning and Namespace Isolation	28
Deployment into Logical Environments	29
4.3 File-Based Data Enrichment (Pull-Based)	30
Supported File Formats and Conversion Rules	30
Field Mapping Utility for Transformation	31
Supported Source Types (SFTP/HTTP)	31
Scheduling and Triggering Options	31
Data Validation and Storage in Postgres	34

Error Handling and Logging	35
Deployment of Pull Jobs to Logical Environments	36
4.4 Mapping Utility	37
Field Association and Mapping	37
One-to-One, One-to-Many, Many-to-One Mapping	41
Constant Value Injection.....	42
Use of Tazama Message Templates.....	42
Output Validation.....	42
Simulation/Dry-Run Mode	43
Versioning and Configuration Management.....	44
User-specific Actions	44
4.5 Data Destination and Storage	44
Writing to Postgres Collections.....	44
Restriction on External Writes.....	45
Dynamic Collection Creation with Schema Mapping	45
Metadata Storage and Management	46
5. Deployment Environments	48
Deployment Across Environments	48
Additional Considerations	51
Implementation Plan.....	51
Package Format	51
Securing the Config Package	52
6. Assumptions and Dependencies	54
7. Glossary	54

Introduction

1.1 Purpose of the Document

This document defines the functional specifications of the Tazama Connection Studio (TCS) for the Tazama platform. It translates the business requirements into detailed technical guidance to support the development, configuration, and deployment of the TCS. The document outlines how the TCS enables data ingestion using schema-driven REST APIs or schedule-based batch jobs. This specification serves as a reference for developers, integrators, and stakeholders throughout the TCS's implementation lifecycle.

1.2 Scope of the System

- Tazama Connection Studio (TCS) is a standalone integration and enrichment module within the Tazama platform.
- Covers configuration and deployment of REST API endpoints for ingestion of messages.
- Handles new message types and custom data elements for financial transactions via TCS-defined schemas.
- Supports mechanisms for both push-based and pull-based data ingestion including file-based enrichment sources.
- Outlines the mapping utility for source-to-target transformation, including one-to-one, one-to-many, and many-to-one relationships.
- Validation, simulation, and version control of mapping configurations using predefined Tazama message templates.
- Structured storage of enriched and mapped data within Postgres collections as part of the Tazama ODS.
- Establishes role-based access control, versioning, and endpoint isolation features for secure TCS operations.
- New endpoints and schemas are deployed across environments using configuration files.
- Logging and observability mechanisms with Tazama's centralized logging framework for traceability and operational monitoring.

Overall Description

2.1 Operating Environment

Tech stack information:

- Frontend: React 18.3 - frontend sessions shall use SameSite=Strict, HttpOnly, Secure cookies.
- Backend: TypeScript (NestJS)
- Database for Tazama Data Model/TCS: PostgreSQL
- IAM: Keycloak
- All components shall be containerized in Docker

System Architecture Overview

3.1 High-Level Architecture

The architecture below distinguishes between two key phases in the lifecycle of the Dynamic Event Monitoring Service (DEMS): Configuration Runtime and Production Runtime.

- **Configuration Runtime** refers to the **administrative process** of defining, securing, and deploying endpoints. This includes user interactions through the Tazama Connection Studio (TCS), authentication via the Auth Service, and persistence of endpoint metadata through the Admin Service into the database.
- **Production Runtime**, by contrast, **represents the live operation** phase where external entities—such as financial institutions, payment switches, or schemes—send real-time messages to the deployed endpoints. These messages are processed by DEMS and routed onward to downstream systems like the Event Director.

The diagram provides a high-level view of the services involved in each phase and the interaction flows that support secure, auditable, and dynamic ingestion management.

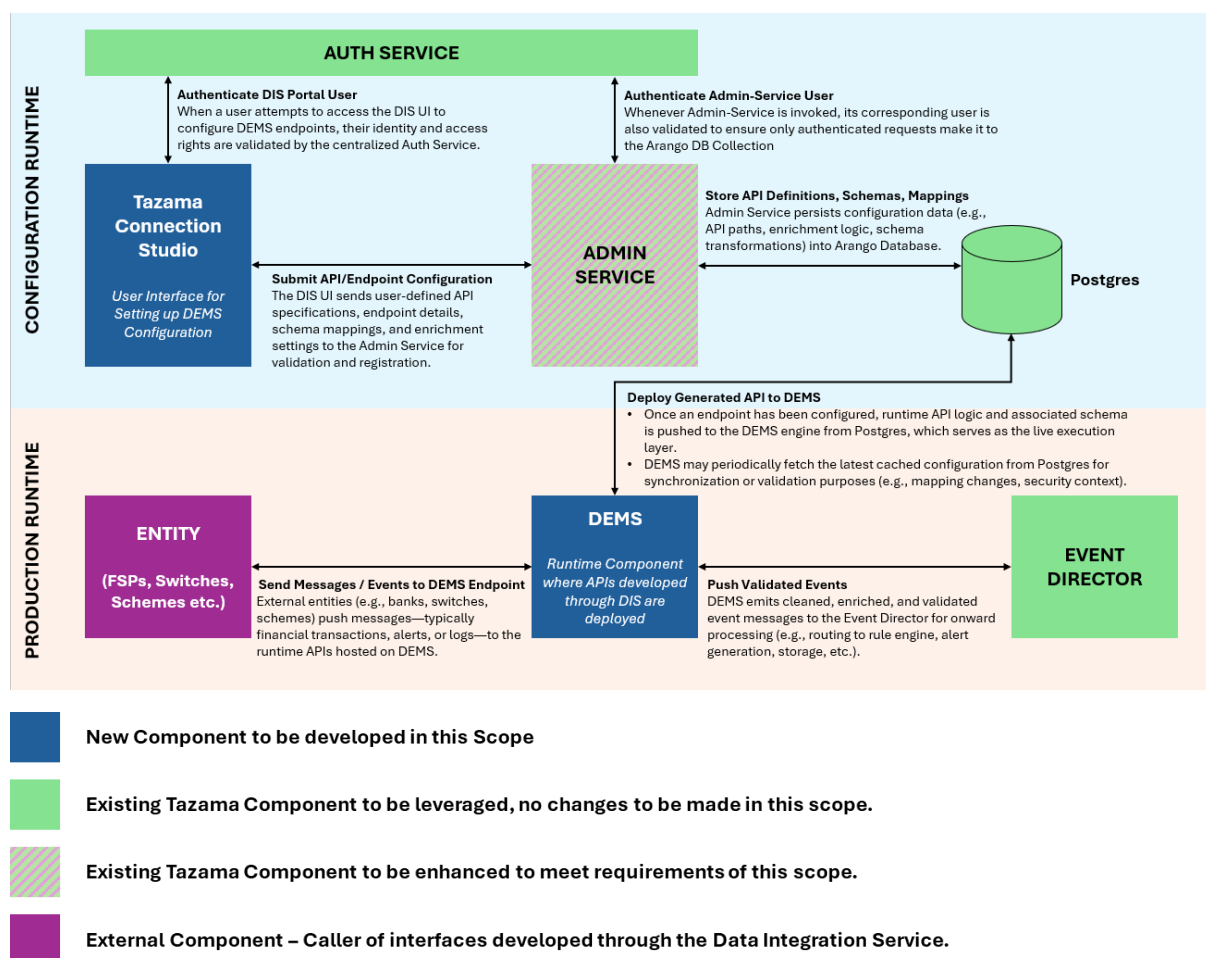


Figure 1. High-level Landscape diagram

Key Components:

- **Auth Service:** A centralized authentication and identity management system (e.g., Keycloak) responsible for:
 - Validating users accessing the TCS Portal
 - Authenticating Admin-Service itself to protect sensitive operations (e.g., writing to the DB)
 - Enforcing role-based access control (RBAC) across services.
- **Tazama Connection Studio (TCS):** Frontend portal and orchestration layer for API publishers to:
 - Define new ingestion APIs (including schema, enrichment rules, endpoint paths)
 - Submit these configurations to the Admin-Service.
 - View the status of deployed ingestion endpoints.
- **Admin Service:** The backend configuration manager that:
 - Accepts API definitions from TCS.
 - Persists endpoint metadata, schemas, and enrichment mappings into Postgres.
 - Optionally handles caching or validation workflows.
- **Postgres:** The centralized metadata store that holds:
 - API definitions
 - Schema mappings
 - Enrichment and transformation logic
 - Configuration version history (if implemented).
 - Exposes APIs for DEMS to pull out the latest deployed configurations.
- **Entity (FSPs, Switches, Schemes, etc.):** An external integrated system (e.g., bank, wallet, switch) that;
 - Sends transaction messages, alerts, logs, or telemetry
 - Authenticates once via Auth Service and includes a valid token with each message
 - Targets DEMS endpoints configured earlier via TCS.
- **DEMS (Runtime Engine):** The execution runtime that;
 - Hosts all deployed ingestion endpoints
 - Validates and processes inbound messages
 - Performs enrichment, transformation, schema validation, and basic sanitization
 - Stores transactions in Postgres
 - Forwards cleaned events to the Event Director for downstream handling
- **Event Director:** A downstream message router or processor that;
 - Receives enriched and validated events from DEMS
 - Routes these events to rule processor

3.2 Configuration Flow

This diagram outlines the end-to-end process of defining and registering a new API endpoint through the TCS (Tazama Connection Studio) interface. The flow is initiated by an API Editor, who authenticates via the Auth Service and proceeds to define the endpoint configuration.

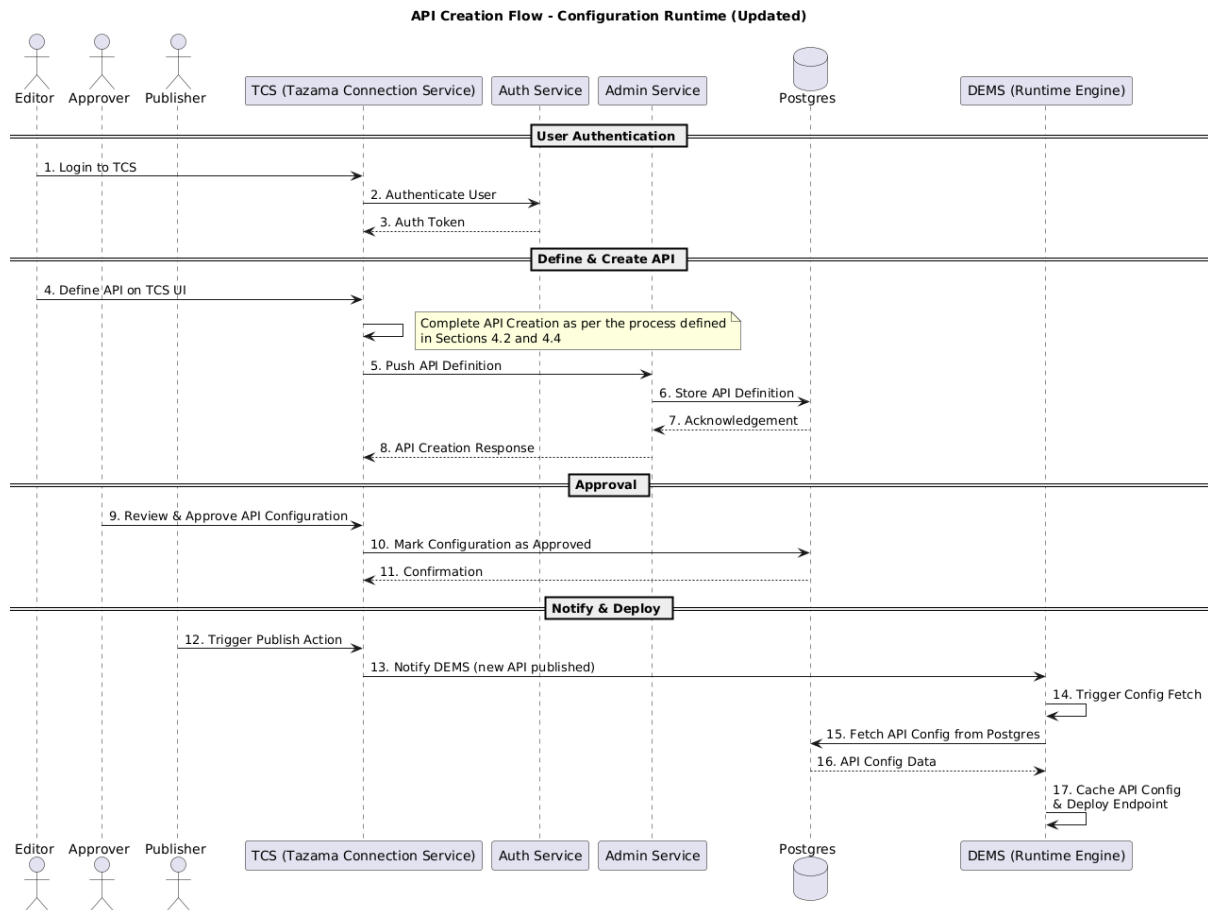


Figure 2. Endpoint Creation Flow Diagram

Key points:

- TCS acts as the orchestration layer, capturing user input and delegating the creation request to the Admin Service.
- The Admin Service persists the configuration in Postgres and confirms the operation back to TCS.
- Upon successful creation, TCS notifies DEMS that a new endpoint has been published.
- DEMS fetches API configuration from Postgres database and maintains a runtime cache

This approach ensures clear separation between configuration workflows and runtime behavior, with a secure and **auditable handoff** between services.

3.3 Transactional Flow

This diagram illustrates the runtime transaction/event flow for entities (such as FSPs, switches, or schemes) integrated with the DEMS engine. It distinguishes between the initial authentication setup and the repetitive transaction processing loop.

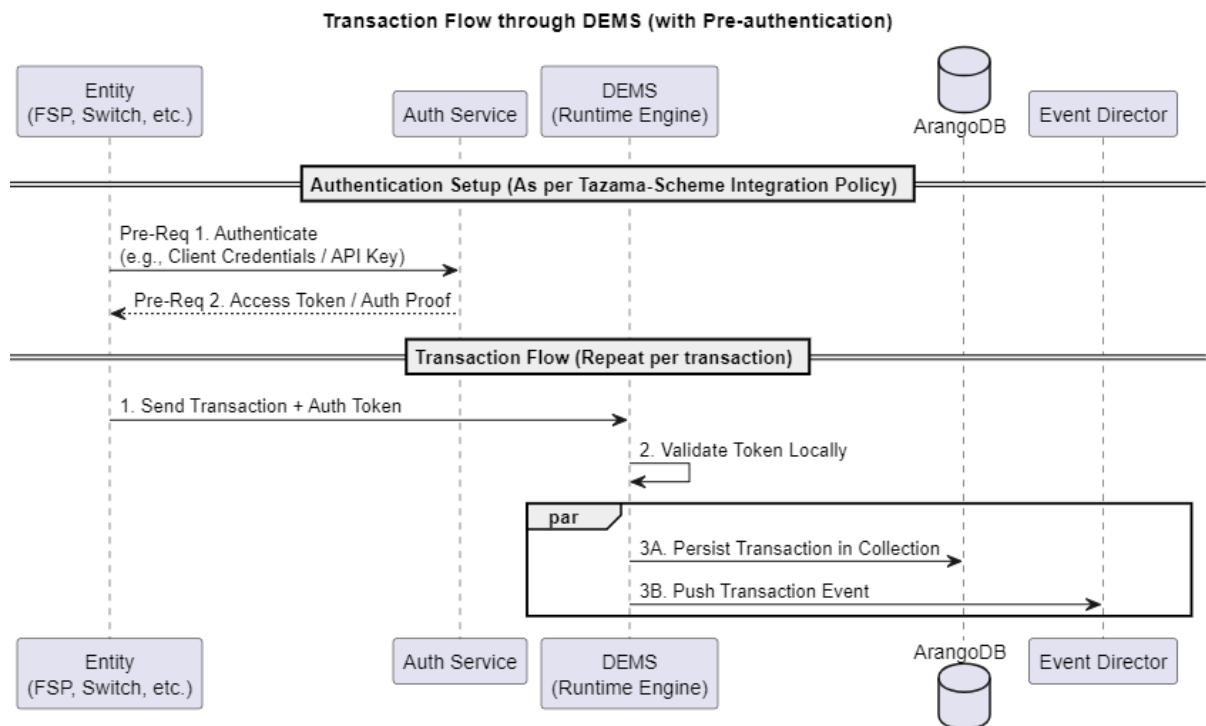


Figure 3. Running Transaction Flow Diagram

- **Authentication Setup:** Prior to initiating transaction requests, each entity must authenticate with the Auth Service (e.g., using client credentials, API key, or mutual TLS). The entity receives an access token or equivalent credential, which is used for all subsequent API calls. This authentication step is performed once per session or token lifecycle, not per transaction.

Note: For demonstration and testing purposes, authentication can be optionally bypassed by setting the `AUTHENTICATED=false` environment variable — consistent with other Tazama APIs.

- **Transaction Flow:** Each transaction sent to DEMS includes the previously issued token for authorization. DEMS validates the token locally (e.g., verifying a JWT signature) without needing to call the Auth Service again.

Upon successful validation:

- The transaction is persisted into Postgres for traceability and storage.
- It is then forwarded to the Event Director component for downstream processing such as typology evaluation, alert generation, or analytics.
- Messages received by an endpoint must be validated against a JSON or XML schema.

- Messages must be mapped or transformed to an internal Tazama JSON payload, the Tazama data model, and a cacheable object.

4. Functional Overview

4.1 General Requirements

Decoupled Operation from DEMS Runtime

The Tazama Connection Studio (TCS) is a standalone tool used to **configure new data flows** into the Tazama platform. It allows users to **define ingestion endpoints**, **apply data transformations**, and **set up enrichment logic** — all without affecting the live DEMS runtime. The outputs from TCS (such as schemas, mappings, and endpoint definitions) are deployed to the DEMS engine, which handles real-time ingestion. **This separation allows integration and implementation partners to onboard both financial and non-financial data into Tazama’s operational data store (ODS) in a safe and controlled manner.**

Feature	Description
Runtime Independence	TCS operates without any runtime dependency on the DEMS engine. No libraries, APIs, or processes from the DEMS are required for its core functions.
Offline Operation	TCS-based configurations (Section 4.2 through 4.5), mappings and transformations (Sections 4.3 and 4.4) can be developed and tested offline or in staging environments before being deployed to DEMS endpoints.
Configuration Artifacts	TCS produces artifacts (API definitions (Section 4.2), mapping configurations, collection schemas) that are deployed to the DEMS to be used at runtime — but the TCS itself does not run in production alongside the DEMS core.
Deployment Artifacts	All outputs of the TCS (API endpoints, enrichment services, Postgres collections) are version-controlled and deployable in configuration bundles/packages.

Runtime Separation

- **DEMS Runtime Scope:**
 - Handles real-time transaction execution for dynamic endpoints.
- Interfaces with Postgres for populating the ingested and transformed data objects into Tazama Collections.
- **TCS Scope:**
 - Manages API schema generation and configuration
 - Handles mapping and data transformation logic creation
 - Connects with enrichment data sources (push/pull)
 - Generates and deploys REST API endpoints under the DEMS framework.

TMS Types

- **Dynamic Event Monitoring Service (DEMS)**

The DEMS ingests incoming transactions and uses the **Catch-all function** to retrieve the corresponding method and endpoint URL from the cache or database. If a matching entry is

found, the transaction proceeds to processing. If not, an error message is returned to the client system, prompting the client to configure the required endpoint through the web portal UI.

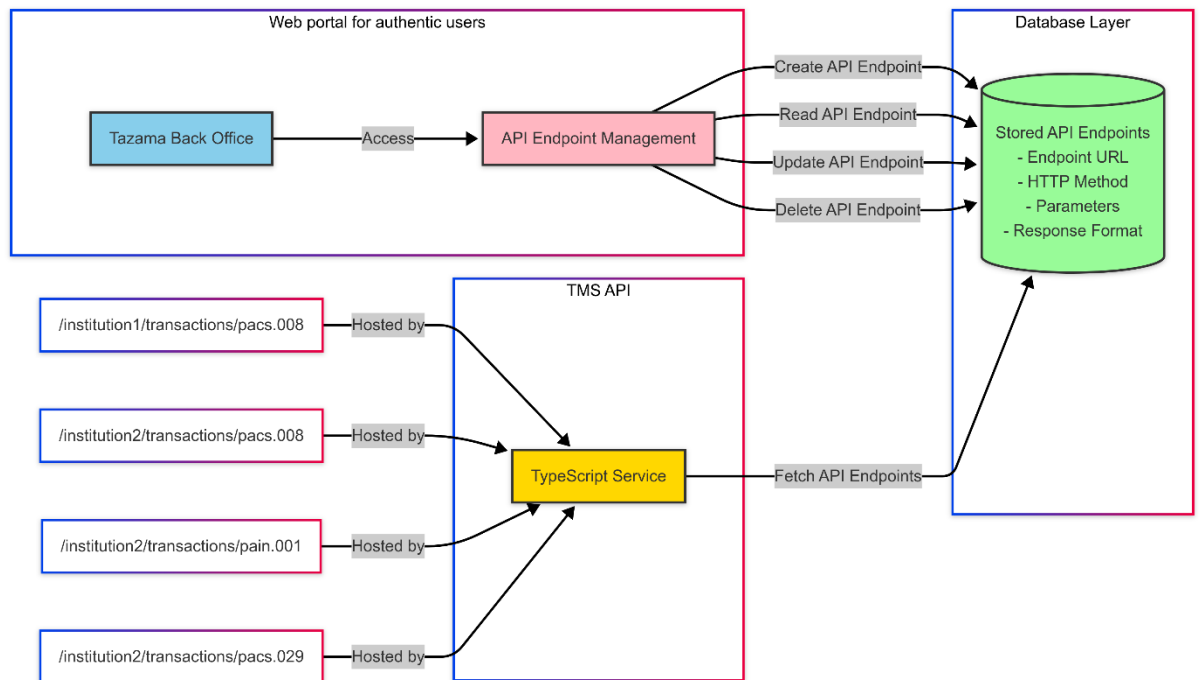


Figure 4. DEMS flow

- **Static TMS**

In the existing static version of the TMS, endpoints are hardcoded. Incoming transactions are processed only if their corresponding endpoint is already defined; any new or undefined endpoints are ignored. To support new endpoints, users must manually generate them through code, making the system less flexible and harder to scale.

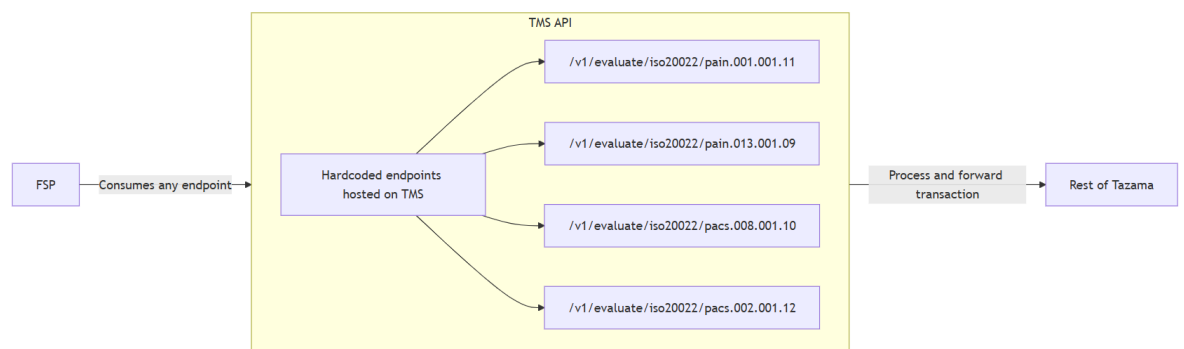


Figure 5. Static TMS flow

Deployment Separation

- The TCS would be a utility to manage the Dynamic Event Monitoring Service. DEMS would be used to configure Transactional Message Ingestion.

- The Dynamic Enrichment API (DEAPI) is expected to largely mirror the functionality of the DEMS API, particularly in terms of supporting dynamic endpoints, data transformation, and database updates. However, unlike the DEMS API, data ingested through the DEAPI is intended for bulk ingestion—regardless of whether it consists of a single record or multiple—and is not meant to be processed solely for evaluation purposes.
- Any deployments to a target environment would be done by packaging artifacts generated by TCS (such as REST endpoints, JSON schema validators, mappings) via exportable packages.

Usage Workflow

1. User launches TCS GUI.
2. Defines a message transformation (e.g., ISO20022 → JSON)
3. Configures enrichment ingestion (via REST, CSV, etc.)
4. Generates API endpoint + Postgres collection definition
5. Performs dry-run or simulation of data flow (refer to Section 4.4 - Simulation/Dry-Run mode)
6. Exports configuration bundle (refer to Section 5 - Deployment Environments)
7. Publisher deploys configuration to target environment
8. The DEMS references active schema and enrichment logic for runtime data validation and enrichment.

Safeguards

- TCS has **no permission** to modify runtime DEMS components.
- TCS actions are sandboxed and produce deployable configurations, no changes in the configurations can be done at runtime.

Limitations and Exclusions

- TCS does not:
 - Interfere with or hook into live DEMS processes
 - Perform runtime message processing
 - Monitor or consume live data streams (e.g., Kafka, message queues)

Logging & Auditing

All TCS actions—such as endpoint generation, data mapping configuration, enrichment source setup, and deployment operations—shall be logged locally in the TCS 's configuration repositories, in alignment with [Tazama's Logging Framework](#). Each log entry must include:

- Timestamp of action
- Action type (e.g., "Endpoint Created", "Mapping Updated")
- Actor identity (user ID or service account)
- Target component or artifact
- Outcome status (success/failure) and error trace (if applicable)

In addition to system logs, the TCS shall produce audit logs that capture traceable user and service interactions with the system for governance and compliance. These logs must include:

- Timestamp of user or service action
- Actor identity (e.g., user ID or service account)
- Action performed (e.g., “Configuration Deployed”)
- Affected entities/fields (e.g., endpoint ID, mapping name)
- Authentication context (if applicable)

Audit and System logs must be sent over TLS, immutable, securely stored for a set period of time before being moved to the Data Warehouse and made accessible for auditing and monitoring purposes.

Assumptions

- Target environments where TCS deploys its outputs (API layer, Postgres, etc.) must be provisioned and reachable
- DEMS runtime expects well-formed JSON messages and schema-conformant enrichment data

Interface Requirements

- **Inputs:** JSON schemas, XML samples, field mappings, CSV files
- **Outputs:** REST endpoint specs, Postgres collection definitions, enrichment job configs
- **Deployment Targets:** DEMS API layer, Enrichment API layer

User Types

The Tazama Connection Studio is designed **for use by System Integrators**, with support for three key roles: Editors, Approvers and Publishers. The TCS provides a role-aware interface that tailors access to features based on each user's responsibilities and technical proficiency. Editors are responsible for creating and configuring endpoints, schemas, and mappings. Approvers, on the other hand, are tasked with reviewing these configurations, approving them, and Publishers are tasked to deploy the finalized packages to the DEMS deployment zone. This streamlined role model ensures a secure, controlled, and auditable integration workflow within the Tazama ecosystem.

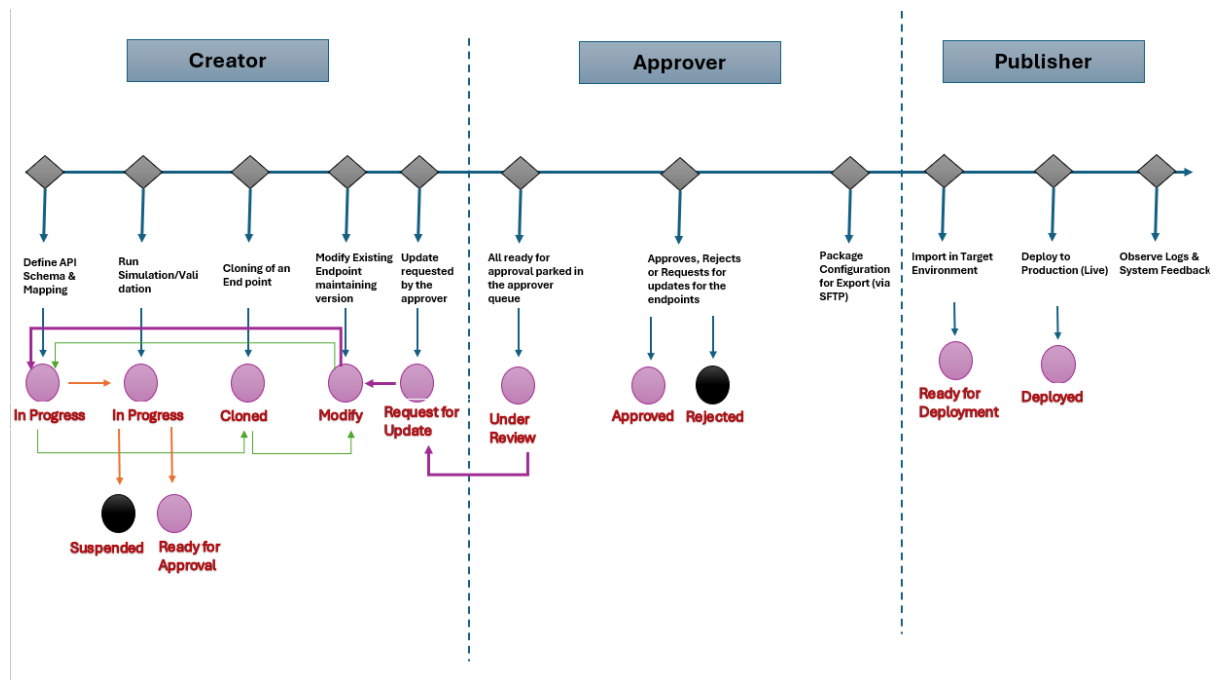


Figure 6. User Types, Responsibilities and State Diagram

User Type Responsibilities

Responsibilities	Creator	Approver	Publisher
Define API Schema & Mapping	Responsible for creating new schemas and mappings	Reviews for completeness	Not Involved
CRON Job for Enrichment	Creates a CRON Job	Approve a CRON Job	Can schedule CRON Jobs
Run Simulation / Validation	Runs dry-run and tests field mappings	May request changes	Not Involved
Validate & Approve Configuration	Not Responsible	Explicitly approves the configuration	Not Responsible
Package Configuration for Export (via SFTP)	Not Responsible	Exports validated configuration (schemas, mappings, CRON)	Not Involved
Import in Target Environment	Not Responsible	Not Responsible	Responsible (Importer role)
Deploy to Production (Live)	Not Responsible	Not Responsible	Triggers publish action, logs status
Observe Logs & System Feedback	Not Responsible	Not Responsible	Monitors simulation logs and runtime behavior, reviews deployment success/failure logs, maintains deployment and infrastructure

Modify / Replace Existing Endpoint maintaining versions	Can initiate updates to endpoint schema or mappings	Must approve updated configuration before redeployment	Manages redeployment
--	---	--	----------------------

System Integrators

1. Expected Skill Level

- Intermediate technical background
- Familiar with API design, schema modeling, transformation logic, and data mapping
- Experience with infrastructure, deployment, and database structures (e.g., Postgres)

2. Primary Functional Access

Functionality	Access Level	Details
API Endpoint Generation	Full	Create, configure, and deploy TCS-generated API endpoints into any pre-defined deployment environment.
Enrichment Job Setup	Full	Define push and pull-based data enrichment jobs, including advanced transformations and scheduling.
Mapping Utility	Full	Define complex mappings, transformations (1:1, 1:N, N:1), constant value injection, simulation and validation.
Postgres Schema Configuration	Full	Create new collections, manage schema conformity, assign destinations for mapped data.
Access Management	Full	Assign and manage Keycloak roles for all TCS-generated endpoints.
Deployment Control	Full	Deployment of endpoint/mapping configurations into logical environments.

4. Additional Privileges

- Modify TCS host configurations
- Manage deployment environment definitions

UX Considerations per User Type

Feature	System Integrators
Schema Editor	Advanced + Raw Mode
Mapping Tool	Full access
Endpoint Generator	GUI

Feature	System Integrators
Simulation Mode	Full (all templates)
Deployment Trigger	All environments

Assumptions

- System integrators are responsible for onboarding new message templates and supporting DFSP requests for advanced configuration.

Data Enrichment Use Cases

The Tazama Connection Studio (TCS) supports the enrichment of transactional data across multiple integration scenarios. This section details the functional capabilities for the primary use cases identified in the BRS.

Use Case 1: Push-Based Data Enrichment via REST APIs

a. Description

External systems may push enrichment data in bulk to a Data Enrichment Endpoint deployed in the Data Enrichment API (DEAPI).

b. Functional Capabilities

- TCS allows creation of enrichment-specific REST endpoints.
- Users can define expected structure via schema or sample message.
- Endpoint stores validated JSON documents into enrichment collections.

c. Approach

- REST endpoints are generated with a custom path (e.g., /TCS/v1/tenantId/enrichment/kycInfo).
- Server-side validation ensures payloads conform to defined schema.
- Data is stored in designated enrichment collections, with appropriate meta-data. (e.g., source system, timestamp, including version control).

The enrichment ingestion process supports two modes of data handling: append (adding new records to existing data) and replace (submitting a new dataset that supersedes the existing one). In "replace" mode, the TCS does not delete existing records directly; instead, it performs an archive-and-append, retaining the previous dataset as a historical snapshot before ingesting the new data.

d. Preconditions

- Enrichment schema must be pre-registered in the TCS.
- Deployment environment must support enrichment endpoint deployment.

Use Case 2: Pull-Based Data Enrichment via Polling or File-Based Inputs

a. Description

The TCS should enable enrichment ingestion by **pulling** data from external systems or files. This supports environments where external systems cannot push data directly.

b. Functional Capabilities

- Support for:
 - Pulling from SFTP/FTP using SSH or password
 - Polling public HTTP GET endpoints returning JSON
 - Reading CSV/TSV files convertible to JSON
- Scheduled ingestion using CRON or manual triggers
- Data transformation and mapping using column-based utilities

c. Approach

- User defines:
 - Source endpoint (URL, SFTP path)
 - Authentication (key, password)
 - Delimiter and field mapping for CSVs
 - CRON expression or manual mode
- TCS transforms and validates each ingested file
- Valid entries are stored in Postgres; failed entries are logged and retained

d. Preconditions

- Pull jobs must be pre-configured and tested in non-production environment
- Source systems must expose readable endpoints or file shares

4.2 API Endpoint Construction and Deployment

The Tazama Connection Studio (TCS) is responsible for generating, validating, and deploying REST API endpoints that serve as ingress points for transactional messages and enrichment data. These endpoints are structured around user-defined schemas and enforce data validation, access control, and logical environment separation. This section provides a breakdown of how endpoints are built, secured, and managed across the platform.

Auto-generation of API Endpoints

The TCS supports auto-generation of REST endpoints using either:

- A **sample payload** (JSON, XML or CSV), or
- A **JSON schema definition**.

Use Case 1: Create New Connection

Step 1: View Connections Queue

The user begins on the Dynamic Endpoint Monitoring Service screen, which is a dashboard for creating and monitoring dynamic API connections. It allows users to search, view, and take actions on various connection configurations, displaying key details such as endpoint path, tenant ID, status (e.g., Ready for Approval, In-Progress, Suspended), and creation time. Users can create new connections, view validation logs, and perform actions on each endpoint, making it a centralized interface for tracking the lifecycle and readiness of integration endpoints.

Note: Links to the prototypes are in the Glossary.

< Back

Dynamic Endpoint Monitoring Service

Jane Doe - Editor

Logout

Search endpoints...

Validation Logs

+ Create New Connection

ENDPOINT PATH	TENANT ID	STATUS	CREATED TIME	ACTIONS
/v1/evaluate/ACM102/iso20022/pacs.008.001.011-transfers	ACM102	Ready for Approval	10/15/2023, 5:00:00 AM	View Actions
/v1/evaluate/FIN345/iso8583/0200-payments	FIN345	In-Progress	11/5/2023, 5:00:00 AM	View Actions
/v1/evaluate/BNK123/iso20022/pacs.002.001.011-transfers	BNK123	Ready for Approval	8/30/2023, 5:00:00 AM	View Actions
/v1/evaluate/GLB789/iso20022/pain.013.001.010-payments	GLB789	Ready for Approval	7/12/2023, 5:00:00 AM	View Actions
/v1/evaluate/PAY456/iso20022/pacs.008.001.011-payments	PAY456	Suspended	11/1/2023, 5:00:00 AM	View Actions
/v2/evaluate/ACM102/iso20022/pacs.008.001.011-transfers	ACM102	Cloned	11/12/2023, 5:00:00 AM	View Actions

Figure 7. Dynamic Endpoints queue

Step 2: Create New Connection and Provide Sample Payload

Upon creating or editing a connection, the user is presented with a form where they input a custom API path and a sample JSON/XML payload. This sample defines the expected structure of incoming data and forms the basis for schema generation.

Dynamic Endpoint Monitoring Service Jane Doe - Editor [Logout]

Edit Endpoint

1 Payload 2 Mapping 3 Simulation 4 Deploy

Transaction Type
Transfers

Sample Payload XML JSON Import File

```
{
  "transaction": {
    "id": "TX12345",
    "amount": 100.5,
    "currency": "USD",
    "status": "pending"
  },
  "customer": {
    "id": "C789",
    "name": "John Doe",
    "email": "john@example.com"
  },
  "items": [
    {
      "id": "ITEM-1",
      "name": "Product A",
      "quantity": 2,
      "price": 45.25
    }
  ]
}
```

Generate Fields

Back Cancel Save Next

Figure 8. New API Endpoints schema generation

Step 3: Configure Mapping and Send for Deployment

After entering the sample payload, the user clicks "Generate Schema" to let the TCS automatically parse and infer field keys, data types, and required flags. The resulting schema is displayed, allowing manual adjustments to data types and marking required fields.

Dynamic Endpoint Monitoring Service Jane Doe - Editor [Logout]

Edit Endpoint

Generated Fields
Define the structure of your JSON schema based on the input data. For each field, specify its data type.

Field Path	Type
transaction	Object
transaction.id	String
transaction.amount	Number
transaction.currency	String
transaction.status	String
customer	Object
customer.id	String

Buttons: Back, Cancel, Save, Next

Figure 9. API Endpoints field configurations

Next, the user navigates to the **Mapping Utility** and uses the option “Add Data Model Extension” to add a new source field in the Data Model which did not exist before, so that mapping can be carried out seamlessly.

Edit Endpoint

1 Payload 2 **Mapping** 3 Simulation 4 Deploy

Field Mapping

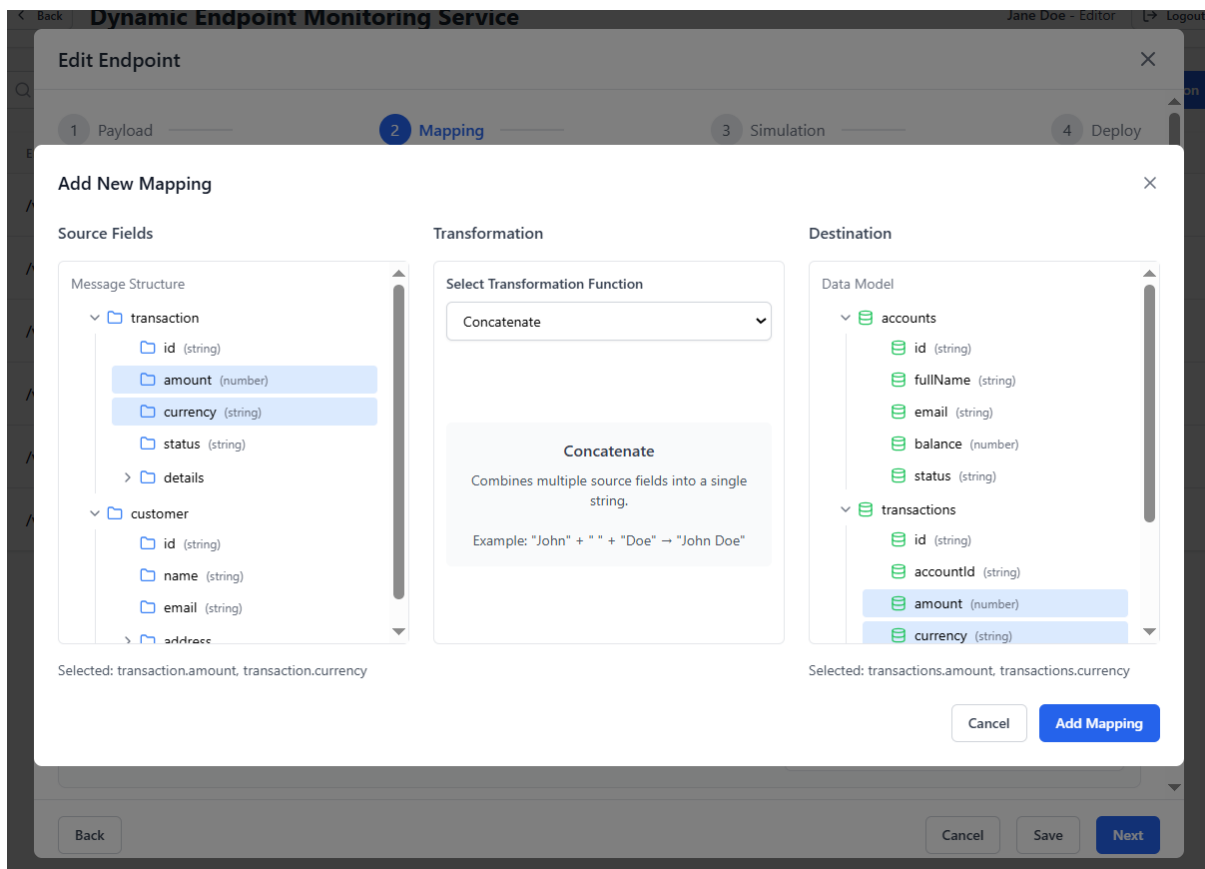
Buttons: Mappings, Data Model Extensions, + Add Data Model Extension

COLLECTION	FIELD	TYPE	REQUIRED	DEFAULT VALUE	ACTIONS
accounts	verificationStatus	string	No	PENDING	✖

About Data Model Extensions
Extensions define new fields to be added to the data model. When this connection is published, these changes will be executed in the database. Make sure to define appropriate default values for required fields.

Buttons: Back, Cancel, Save, Next

Now the user shall define all field mappings between the source structure and Tazama's operational model.



Next, the user runs a simulation on a test payload to verify that the configuration and mapping work as expected.

Dynamic Endpoint Monitoring Service Jane Doe - Editor [Logout]

Edit Endpoint

1 Payload 2 Mapping 3 **Simulation** 4 Deploy

Test Mapping

Test Payload [Import Test File](#)

```
{
  "customer": {
    "id": "C789",
    "name": "John Doe",
    "email": "john@example.com"
  },
  "items": [
    {
      "id": "ITEM-1",
      "name": "Product A"
    }
  ]
}
```

[Run Simulation](#)

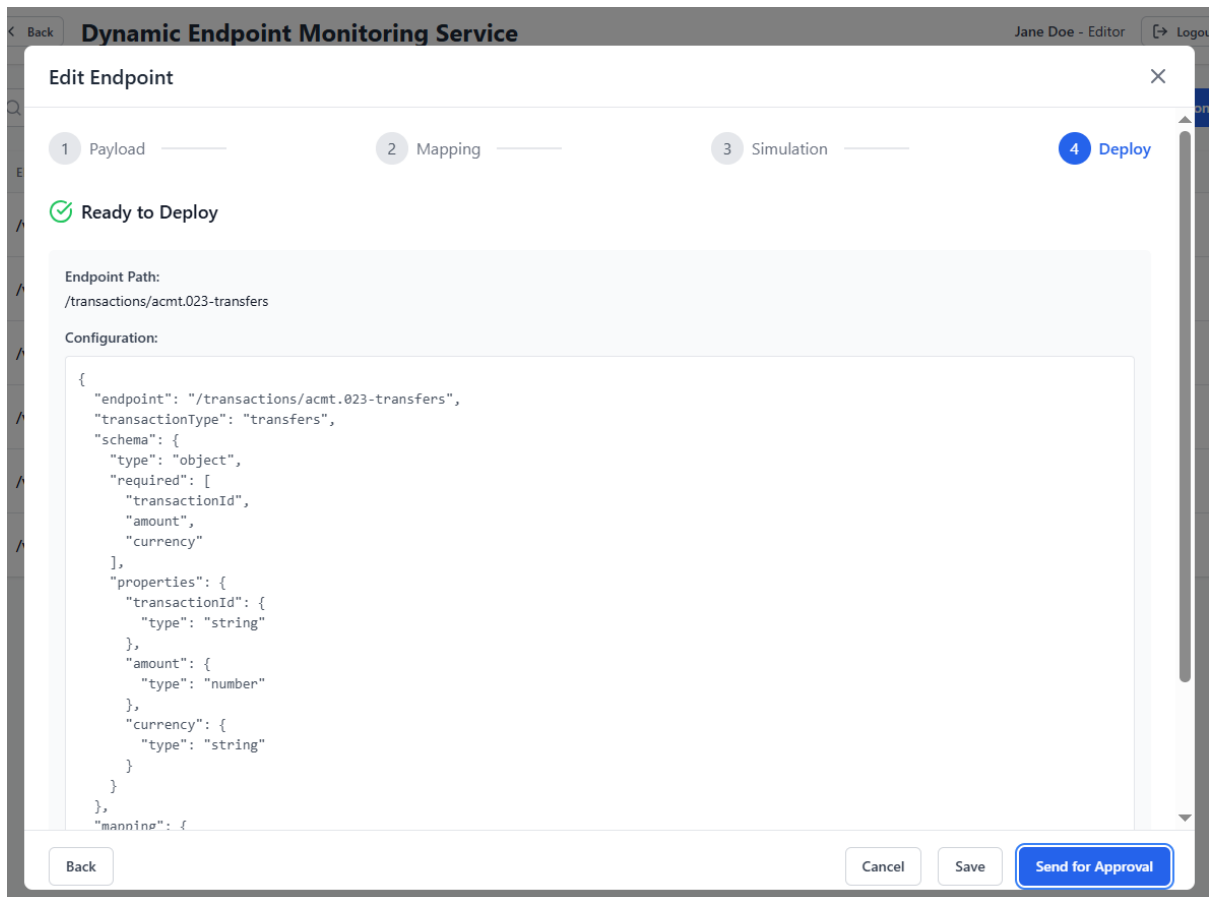
✓ Validation Passed
All fields have been successfully mapped and validated.

Transformed Output:

```
{
  "id": "123456",
  "value": 1000,
  "currencyCode": "USD",
  "currencySymbol": "USD"
}
```

[Back](#) [Cancel](#) [Save](#) [Next](#)

Once the connection configuration is reviewed and finalized, the user clicks "Send to Approver" to then initiate the review of the configuration on the Approvers end, before the configuration package is deployed on the target environment via SFTP.



Dynamic Endpoint Monitoring Service

Jane Doe - Editor [Logout]

Edit Endpoint

1 Payload 2 Mapping 3 Simulation 4 **Deploy**

✓ Ready to Deploy

Endpoint Path:
/transactions/acmt.023-transfers

Configuration:

```
{
  "endpoint": "/transactions/acmt.023-transfers",
  "transactionType": "transfers",
  "schema": {
    "type": "object",
    "required": [
      "transactionId",
      "amount",
      "currency"
    ],
    "properties": {
      "transactionId": {
        "type": "string"
      },
      "amount": {
        "type": "number"
      },
      "currency": {
        "type": "string"
      }
    }
  },
  "mapping": {
```

Back Cancel Save **Send for Approval**

Next Steps:

- The schema is submitted via TCS GUI.
- Upon approval and deployment of the schema, below will be auto-generated:
 - Ingest endpoint
 - Server-side validation logic
 - Corresponding Postgres collection in a database. It should be configurable in the TCS backend that which database the collection needs to be created in. For now, this is transactionHistory, but in the future this may be changed. This collection will serve as the initial "persist-as-received" storage layer for incoming messages. Additionally, new messages will be integrated into the new data model also known as Pseudonyms database.

Use Case 2: Cloning and Modifying Previously Created Connection

This involves enhancing a message already supported by DEMS with additional data fields or metadata.

Step 1: Select and Clone

The user selects an existing connection (e.g., /v1/evaluate/tenantId/iso20022/pacs.008.001.011-payments) from the DEMS queue.

[< Back](#) **Dynamic Endpoint Monitoring Service** Jane Doe - Editor [Logout](#)

[Validation Logs](#) [+ Create New Connection](#)

ENDPOINT PATH	TENANT ID	STATUS	CREATED TIME	ACTIONS
/v1/evaluate/ACM102/iso20022/pacs.008.001.011-transfers	ACM102	Ready for Approval	10/15/2023, 5:00:00 AM	View Actions
/v1/evaluate/FIN345/iso8583/0200-payments	FIN345	In-Progress	11/5/2023, 5:00:00 AM	View History Suspend Endpoint Clone Endpoint
/v1/evaluate/BNK123/iso20022/pacs.002.001.011-transfers	BNK123	Ready for Approval	8/30/2023, 5:00:00 AM	View Actions
/v1/evaluate/GLB789/iso20022/pain.013.001.010-payments	GLB789	Ready for Approval	7/12/2023, 5:00:00 AM	View Actions
/v1/evaluate/PAY456/iso20022/pacs.008.001.011-payments	PAY456	Suspended	11/1/2023, 5:00:00 AM	View Actions
/v2/evaluate/ACM102/iso20022/pacs.008.001.011-transfers	ACM102	Cloned	11/12/2023, 5:00:00 AM	View Actions

Step 2: Modification

Upon cloning the connection, a new connection with the status “Cloned” shall be created, indicating an updated version in the endpoint path. Once the user begins editing the cloned connection, the status changes from “Cloned” to “Modify”, and the user will be able to make edits as needed.

To save and send the new version for deployment, the same steps as detailed in [Step 3](#) of Use Case 1 shall be followed.

[< Back](#) **Dynamic Endpoint Monitoring Service** Jane Doe - Editor [Logout](#)

[Edit Endpoint](#)

[Generate Fields](#)

Generated Fields
Define the structure of your JSON schema based on the input data. For each field, specify its data type.

Field Path	Type
transaction	Object
Field Path	Type
transaction.id	String
Field Path	Type
transaction.amount	Number
Field Path	Type
transaction.currency	String
Field Path	Type
transaction.status	String

[< Back](#) [Cancel](#) [Save](#) [Next](#)

Supported Formats

- **Input:** JSON schema, XML samples
- **Output:** OpenAPI 3.0 YAML/JSON, JSON schema validator, JSON-to-Arango mapping

Restrictions

- Only structured data payloads are supported
- Schemas must not contain recursive definitions or unsupported types (e.g., binary)

Message Pre-validation:

Prior to message ingestion, the DEMS performs attribute-level validation on each incoming message. Specifically, it checks for the presence of three key attributes: **Debtor**, **Creditor**, and **Amount**. This is to create a transaction relationship of the incoming message in the existing data model. The DEMS is configured to **allow messages that contain at least one or two of these attributes**. Messages containing these identifiers are permitted to pass through for further processing, while those containing none of the identifiers shall be stored in a separate, unrelated data model.

Deployment into DEMS and Data Enrichment Layers

TCS distinguishes between two deployment targets:

a. DEMS

- Used for ingesting financial and non-financial transaction messages such as Pain.001 or Acmt.023.
- Used to change current endpoint for an existing message to add new field.

These endpoints:

- Are deployed to the main DEMS API gateway
- Trigger downstream processing in the **Event Director**, with the decision to trigger an evaluation being left to the Event Director. To initiate the evaluation, the message must include a "TxTp" field with a unique value (e.g., "TxTp": "pacs.008.001.10"), which is used by the Event Director to look up the relevant network map and determine how the message should be routed for evaluation.
- Enforce strict schema validation and mapping to operational collections

b. Enrichment Layer(DEAPI)

Used for ingesting enrichment data (e.g., KYC info, risk scores). These:

- Do not invoke real-time transaction flows
- Write to enrichment-specific Postgres database
- Are typically called via scheduled batch systems or CRON jobs, However, functionality will also be provided for manual or system-driven execution where applicable.

These artifacts:

1. Are generated by TCS to be sent to the DEAPI.
2. Artifacts are published to:
 - API gateway routing config
 - Postgres schema registry
 - Deployment logs and endpoint metadata registry

All deployments require that target environments are pre-provisioned and reachable.

Custom API Paths

Users may define the URL path structure of generated endpoints at creation time. TCS validates and enforces naming conventions.

Structure

Example path: `v1/dfsp/{dfsp_id}/pacs009/incoming`

Constraints

- Must begin with `/TCS/v{version}/`
- Must not conflict with existing system APIs or reserved paths

- Paths are validated against registry and normalized (lowercase, kebab-case enforced)

Path Storage

Custom paths are stored as part of the endpoint configuration artifact and versioned. Endpoint invocation logs use the path for audit traceability.

Role-Based Access Control via KeyCloak

The Tazama Connection Studio (TCS) integrates with **Keycloak** as its identity and access management (IAM) system to enforce secure role-based access to all features, APIs, and endpoints deployed. This integration enables fine-grained access control across users and environments.

Supported Role Types

- `internal_admin` — Full control and access

Realm-Based Structure and Environment Isolation

Each deployment environment (e.g., Development, UAT, Production) shall be integrated with Keycloak. Due to security and configuration specificity, Keycloak configuration must be performed manually per environment.

Internal Admin Role and Institutional Configuration

The **Internal Admin Role** is typically granted to trusted system integrators or administrators. They have administrative access to perform the following operations:

- **Create and manage tenants (DFSPs)**
- **Create users** for the system integration roles
- **Define tenant-specific roles** like: editor, approver and publisher
- **Assign access scopes** by mapping roles to UI features and backend permissions

Server-Side Validation

Each generated endpoint includes a **schema validation layer** that enforces payload structure, types, and required fields before processing.

Validation Engine

- JSON: Uses a compiled JSON schema validator (e.g., `ajv`, `jsonschema`, `z-schema`)
- XML: Converted to JSON and validated post-transformation

Validation Rules Enforced

- Field presence (`required`)
- Content type of the payload (XML, JSON, CSV)
- Type constraints (`string`, `integer`, `array`, etc.)
- Format constraints (e.g., date-time, regex patterns)

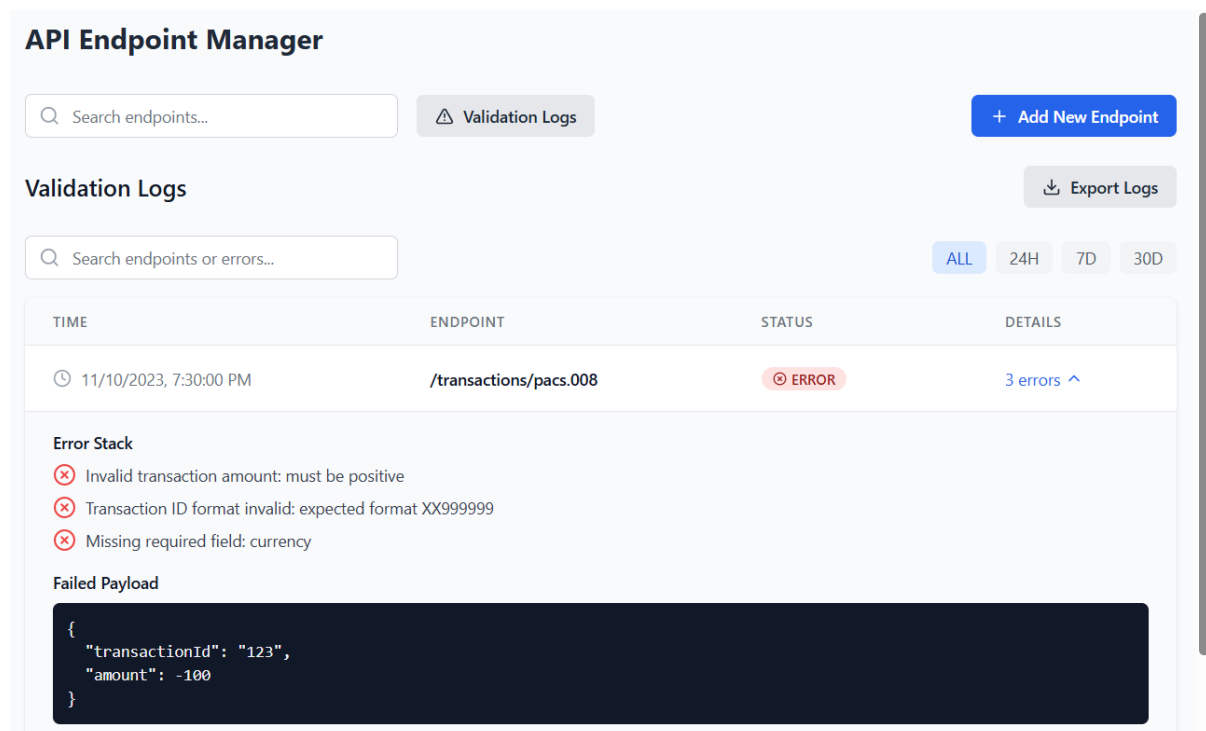
- Enum values
- Nested object structure validation
- Size limit for incoming payloads (e.g. >200MB)

Error Handling

- Invalid input results in:
 - HTTP 400 Bad Request
 - JSON body: {"error": "Validation failed", "details": [...]}

Logging

- Payloads failing validation are logged with timestamp, endpoint, and validation error stack
- Logs can be reviewed via TCS GUI or exported as audit reports



API Endpoint Manager

Search endpoints... Validation Logs + Add New Endpoint

Validation Logs Export Logs

Search endpoints or errors... ALL 24H 7D 30D

TIME	ENDPOINT	STATUS	DETAILS
11/10/2023, 7:30:00 PM	/transactions/pacs.008	ERROR	3 errors ^

Error Stack

- Invalid transaction amount: must be positive
- Transaction ID format invalid: expected format XX999999
- Missing required field: currency

Failed Payload

```
{
  "transactionId": "123",
  "amount": -100
}
```

Figure 19. API Endpoints validation logs

Endpoint Versioning and Namespace Isolation

To avoid schema conflicts and enable parallel development, endpoints are versioned and isolated by namespace.

Versioning

- API version embedded in path: /TCS/v1/..., /TCS/v2/...
- Config artifacts tagged with v1, v2, etc.
- Backward compatibility maintained by preserving older endpoint versions

Namespace Isolation

- DFSP-specific endpoints are deployed under /TCS/v1/dfsp/{dfsp_code}/...

- System integrators may use `/TCS/v1/integrator/...` or `/TCS/v1/admin/...`
- Enforced at deployment time and validated in routing table

Artifact Management

- Each version of an endpoint includes:
 - OpenAPI spec
 - Mapping logic
 - Role binding
 - Deployment history (timestamp, user, target environment)

Deployment into Logical Environments

TCS supports deployment into multiple logical environments, each representing a distinct phase of the development lifecycle.

Available Environments

- **DEV** — Used for local/integration testing
- **QA/UAT** — Used for validation and acceptance testing
- **PROD** — Live production ingestion

Deployment Mechanics

- User creates configuration package for export
- Publisher/ Deployer validates and deploys package on the target environment (Dev, QA, Production)
- TCS performs endpoint registration
- Deployment outcome (success/failure) is logged

Deployment Logging

- Logs include:
 - User ID
 - Timestamp
 - Endpoint path
 - Version
 - Target environment
 - Deployment result

Add New Endpoint
×

1 Payload
2 **Schema**
3 Mapping
4 Simulation
5 Deploy

Schema Generator
<> Raw Edit
Generate Schema

transactionId	^[A-Z]{2}\d{10}\$	string	☒
amount	0	number	☒
currency	USD,EUR,GBP	string	☒

+ Add Field

Back
Cancel
Next

4.3 File-Based Data Enrichment (Pull-Based)

The TCS supports ingesting enrichment data via REST API through pull-based mechanisms. This functionality is used to onboard structured data (such as customer profiles, blacklist entries, KYC metadata, or geo-tagged risk data) that augments fraud and risk monitoring.

Supported File Formats and Conversion Rules

The TCS supports ingestion of structured text files that either:

- Are already in valid JSON format
- Can be transformed into JSON from tabular formats (e.g., CSV, TSV)

Sample CSV Input:

```
customerId,fullName,dateOfBirth,kycStatus
C001,Jane Doe,1985-09-10,Verified
C002,John Smith,1990-02-14,Pending
```

Sample Output JSON:

```
[
  {
    "customerId": "C001",
    "fullName": "Jane Doe",
    "dateOfBirth": "1985-09-10",
    "kycStatus": "Verified"
  },
  {
    "customerId": "C002",
    "fullName": "John Smith",
    "dateOfBirth": "1990-02-14",
    "kycStatus": "Pending"
  }
]
```

Field Mapping Utility for Transformation

To ingest tabular files, TCS provides a GUI utility to configure column-to-field mappings.

User Configuration Options

- Column-to-JSON field mapping
- Delimiter selection (comma, tab, pipe)
- Default values for missing fields
- Field transformation rules, e.g.:
 - Concatenate: `firstName + " " + lastName` → `fullName`

Supported Source Types (SFTP/HTTP)

TCS supports pulling files from the following source types:

A. SFTP/FTP Servers

- Auth via password or SSH key
- Configurable remote path and filename pattern (e.g., `daily_*_kyc.csv`)

B. HTTPS GET Endpoints

- Used when partner exposes a file or JSON data at a public or protected URL

Note: Checksum shall be performed for ensuring that the files are not tampered with.

Scheduling and Triggering Options

Data ingestion can be triggered in two modes:

A. Scheduled (CRON)

- CRON expressions define execution interval

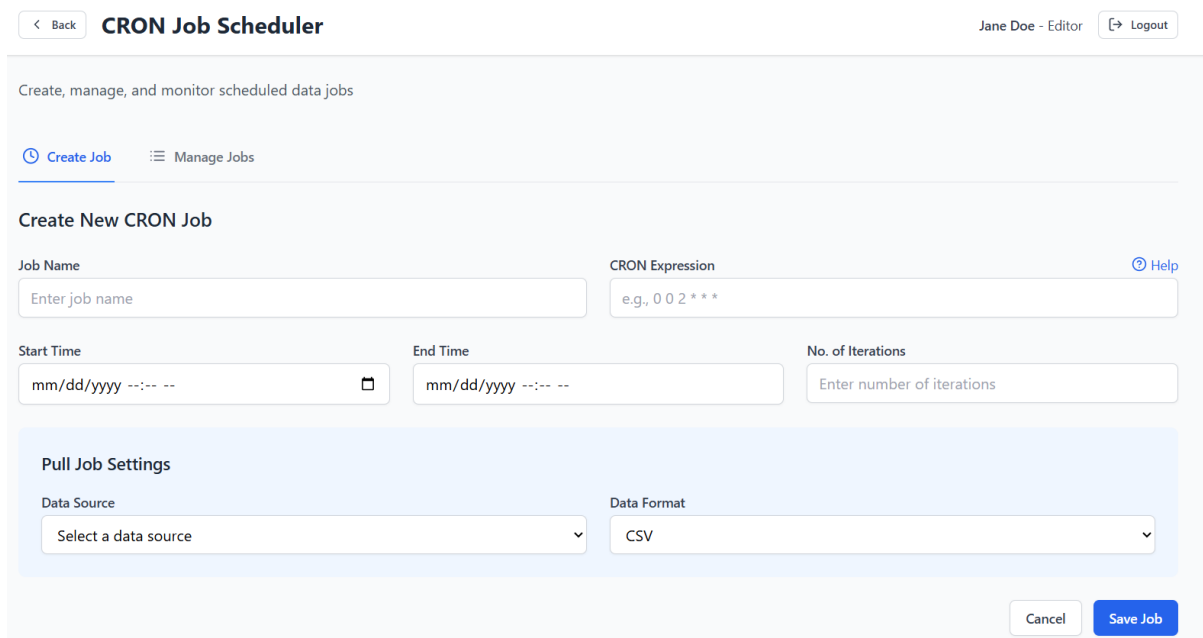
Minute	Hour	Day of the Month	Month	Day of the Week	CRON Expression
0	*/30	*	*	*	0 */30 * * *

- Examples:

- 0 */30 * * * * → Every 30 minutes
- 0 0 2 * * * → Daily at 2:00 AM

- TCS generates a CRON-based job runner script per pull job

The user can also set up a scheduler job from UI. User can select the department, the time period, and other details to help schedule a push-based activity:



The screenshot shows the 'CRON Job Scheduler' interface. At the top, there's a navigation bar with a 'Back' button, the title 'CRON Job Scheduler', and a user profile 'Jane Doe - Editor' with a 'Logout' button. Below the navigation bar is a header section with the text 'Create, manage, and monitor scheduled data jobs' and two tabs: 'Create Job' (active) and 'Manage Jobs'. The main section is titled 'Create New CRON Job'. It contains several input fields: 'Job Name' (placeholder: 'Enter job name'), 'CRON Expression' (placeholder: 'e.g., 0 0 2 * * *', with a 'Help' icon), 'Start Time' (placeholder: 'mm/dd/yyyy --:-- --'), 'End Time' (placeholder: 'mm/dd/yyyy --:-- --'), and 'No. of Iterations' (placeholder: 'Enter number of iterations'). Below these fields is a 'Pull Job Settings' section with two dropdown menus: 'Data Source' (placeholder: 'Select a data source') and 'Data Format' (placeholder: 'CSV'). At the bottom right, there are 'Cancel' and 'Save Job' buttons.

Figure 21. CRON Job scheduler

-
-

Job Name

- **Purpose:** User-defined identifier for the CRON job (e.g., DailyDataPull, WeeklyReportSync).

Feature: Validation to ensure uniqueness; support for naming conventions.

- **Purpose:** Defines when the CRON job should start executing.

- **Description:** A date-time picker field that allows users to specify the starting date and time for the job's first execution.
- **Expected Values:** Date-time format (e.g., 2025-04-30 14:00).
- **Behavior:** Once set, the CRON job will begin running at the specified start time based on the CRON expression.
- **No. Of Iterations**
- **Purpose:** Limits how many times the job should run.
- **Feature:** Useful for temporary or trial runs; mutually exclusive with End Time.

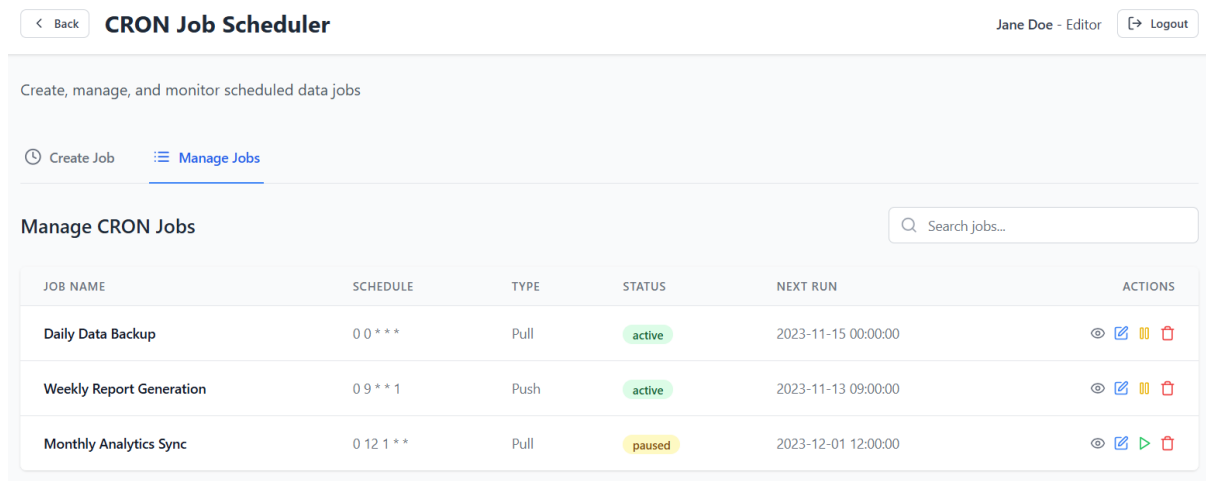
End Time

- **Purpose:** The date/time after which the job should stop running. This is auto-calculated based on **Start Time** and **Number of Iterations**.
- **Feature:** Optional field; may default to 'no end' if left blank. Becomes read-only once CRON and iteration count are provided. Automatically computes the date/time of the final scheduled run.

•

B. CRON Job Management

CRON jobs within the TCS are managed through a centralized management interface.



The screenshot shows the 'CRON Job Scheduler' interface. At the top, there's a 'Back' button and the title 'CRON Job Scheduler'. On the right, it says 'Jane Doe - Editor' and a 'Logout' button. Below the title, it says 'Create, manage, and monitor scheduled data jobs'. There are two tabs: 'Create Job' and 'Manage Jobs', with 'Manage Jobs' being the active tab. Below the tabs, there's a search bar labeled 'Search jobs...'. The main section is titled 'Manage CRON Jobs' and contains a table with the following data:

JOB NAME	SCHEDULE	TYPE	STATUS	NEXT RUN	ACTIONS
Daily Data Backup	0 0 * * *	Pull	active	2023-11-15 00:00:00	  
Weekly Report Generation	0 9 * * 1	Push	active	2023-11-13 09:00:00	  
Monthly Analytics Sync	0 12 1 * *	Pull	paused	2023-12-01 12:00:00	  

Figure 22. CRON Job management

The following functionalities are supported for managing CRON jobs:

Viewing Existing CRON Jobs

- **Description:** Users can view all the CRON jobs they have created in a **list format**. Each job's details, including its execution schedule, status, and next run time, are displayed in the table.

- **Behavior:** The CRON job list is sortable by different fields such as Job Name, Next Run Time, or Status. Users can filter or search through the list to find specific jobs.

Viewing Job History

- **Description:** A history panel shows past executions of the CRON job, including their results, duration, and any errors encountered during execution.
- **Displayed Fields:**
 - Execution Timestamp
 - Job Name
 - Execution Duration
 - Status (Success, Failure)
 - Error Logs (if any)
- **Behavior:** Users can click on any historical job entry to view detailed logs of the job execution. Logs include step-by-step execution details and any failure messages.

Managing Jobs (Edit / Delete)

- **Description:** Users can manage their CRON jobs by editing or deleting them.
 - **Edit:** Clicking the edit button brings up the job configuration screen, where users can modify the CRON expression, job type, or schedule.
 - **Delete:** Clicking the delete button will remove the job from the system. A confirmation dialog ensures that this action is intentional. Only the Admin shall have this privilege.

C. CRON Job Security and Access Control

- **Role-Based Access:** CRON job management is restricted by user roles defined in Keycloak. Only users with the appropriate permissions (e.g., Integrator role) can create, edit, or delete jobs.
- All CRON job SFTP credentials shall be encrypted and stored in Postgres.

Audit Logging: All actions related to CRON job creation, editing, deletion, and execution are logged for auditing purposes. This includes who created the job, when it was executed, and whether it succeeded or failed.

D. Manual Trigger

- GUI button to run ingestion ad hoc
- Useful for testing, dry runs, or one-time uploads

Data Validation and Storage in Postgres

After transformation, each JSON record is validated and stored.

Validation Process

- Field types, formats, and required keys are enforced via a JSON schema
- Optionally allows soft schema (warnings logged, but not rejected)

Storage Behavior

- All validated entries are inserted into a the relevant Postgres collection, with the following configurable options:
 - **Replace/Overwrite:** Update existing records with matching keys.
 - **Append:** Add new records without modifying existing ones.
 - **Purge and Write:** Clear the collection before inserting new records.
 - **Archive and Write:** Archive the existing record in the collection and write a new one.
- Collection is auto-created by TCS if not found
- Each entry includes metadata fields:

```
{
  "_meta": {
    "source": "sftp://partner.gov/kyc.csv",
    "ingestedAt": "2025-04-09T09:00:00Z",
    "jobId": "pull_kyc_data"
  }
}
```

Error Handling and Logging

TCS includes robust error management to handle and surface issues during ingestion.

File-Level Errors

- Connection refused
- Auth failure
- File not found
- Invalid format

→ **Result:** Job is marked as failed, file is retained, and error is logged.

Record-Level Errors

- It will be kept configurable at the job configuration level that whether in this case file level errors need to be logged or record level errors need to be logged.
- Malformed lines
- Schema validation failure
- Transformation error (e.g., invalid date format)

→ **Result:**

- Invalid rows skipped, logged with reason and line number
- Valid records still inserted

Deployment of Pull Jobs to Logical Environments

Each pull job is deployable to a specific logical environment: e.g. DEV, UAT, or PROD.

Deployment Behavior

- A job can only run in environments where:
 - Its source config exists
 - The target Postgres is reachable and compatible with the source environment

Access Restrictions

- System integrators can export configuration package from one environment through configuration packages for them to be imported in the destination environment (refer to Section 5 – Deployment Environments)

4.4 Mapping Utility

Mapping Utility is a core component of the Tazama Connection Studio responsible for transforming incoming data into Tazama-compliant message formats. It enables users (system integrators, editors) to define how fields from an incoming message payload (source) should be associated with the expected format (destination), based on standardized message templates (e.g., ISO 20022, ISO 8583). This configuration governs how transaction data is normalized, validated, and persisted within the Tazama operational data store.

Field Association and Mapping

Functional Behavior:

- Users interact with a GUI-based interface to define mappings between source payload fields and destination message fields.
- Source fields are extracted from the payload's hierarchical structure (JSON paths, XML tag hierarchy if applicable).
- Destination fields are selected from previously created data model stored within Tazama. Below is an extract from the possible Tazama Data elements derived from existing rules in the system, on which the fields from the API shall be mapped. A possible destination the user can map source fields to is the Valkey cache.
- Users must be able to define new target data structures and specify the corresponding Postgres collections for storing mapped data, in addition to using existing message templates.

Type	Source	Data Element (TMS API)	Format	Derived Parameters Breakdown	Description	Field in Rules Code (TS Code)	Stage	Input to PPA	TMS API - Txn Type or Event Type	Output from PPA to TMS
Payer Info	Payment System	Payer Name	A	N/A	Name of Payer (Debtor)		POST /QUOTES	payer.creditParty.Name	PAIN.001	/Document/CstmrCdtTrfInitn/PmtI
Payee Info	Payment System	Payee Name	A	N/A	Name of Payee (Creditor)		PUT /QUOTES	Payee.debitParty.Name	PAIN.013	/Document/CstmrCdtTrfInitn/PmtI
Payer Info	Payment System	Debtor Account ID	ANS	N/A	Account ID of the Debtor/Payer	dbtrAcctId	POST /QUOTES			
Payee Info	Payment System	Creditor Account ID	ANS	N/A	Account ID of the Creditor/Payee	cdtrAcctId	POST /QUOTES			
Txn Info	Payment System	Current Transaction Time Frame	DateTime	N/A	The creation date and time of the current transaction	currentPacs002TimeFrame	POST /QUOTES	CreDtTm	PAIN.001	/Document/FitToFICstmrCdtTrf/Grp
Txn Info	Payment System	Original End-to-End ID	ANS	N/A	The unique identifier for the transaction.	EndToEndId	POST /QUOTES	transactionId	PAIN.001	/Document/CstmrCdtTrfInitn/PmtI
Txn Info	Configuration Parameter	Tolerance	N	N/A	The allowable tolerance percentage for comparing transaction amounts.					
Txn Info	Payment System	Transaction Status	A	N/A	The status of the current transaction (e.g., "ACCC" for accepted). There will be other statuses that will imply Success	req.transaction.FitToFIPmtSts.TxnInfA	PUT /TRANSFERS		PACS.002	/Document/FitToFIPmtStsRpt/TxnInfA
Txn Info	Configuration Parameter	Evaluation Interval Time	DateTime	N/A	The interval time used for evaluating transaction activity.					
Txn Info	Payment System	Transaction Amount	N	N/A	Also in Rule 20 and 48 (re-check) The amount of the current transaction	pac008.Amt	POST /QUOTES	amount	PAIN.001	/Document/CstmrCdtTrfInitn/PmtI
Txn Info	Configuration Parameter	Commission	N	N/A	The commission percentage to be applied to the target amount					
Payer Info	Payment System	Date of Birth	DateTime	N/A	The birth date of the debtor retrieved from the transaction history	pac008.FitToFICstmrCdt.CdtTrfTxnf	POST /QUOTES			
Payer Info	Tazama	Debtor Age	N	N/A	The age of the debtor at the time of the transaction.	f.Dbtr.Id.Prvtld.DtAndPlcOfBirth.Birt	TMS			
Txn Info	Configuration Parameter	Minimum Number of Transactions	N	N/A	Calculated by Tazama based on Date of Birth & the Timestamp when transaction landed in TMS					
Txn Info	Configuration Parameter	Maximum Radius	N	N/A	The minimum number of historical transactions required to apply Benford's Law.					
Payer Info	Payment System	Debtor ID	ANS	N/A	The maximum radius for the Haversine distance calculation	req.DataCache.dbtrId	POST /QUOTES	payer.debitParty	PAIN.001	/Document/CstmrCdtTrfInitn/PmtI
Payee Info	Payment System	Creditor's ID	ANS	N/A	Unique identifier for the debtor involved in the transaction.	req.DataCache.cdtrId	PUT /QUOTES	payee.creditParty	PAIN.013	/Document/CstmrCdtTrfInitn/PmtI

Figure 23. Tazama Conceptual Data Elements

Technical Design:

- Support for nested structures using JSONPath notation or XPath (in the case of XML).
- UI shall display a hierarchical view of both source and destination schemas.
- Field mapping must support basic type compatibility (e.g., number-to-string coercion, date format alignment).

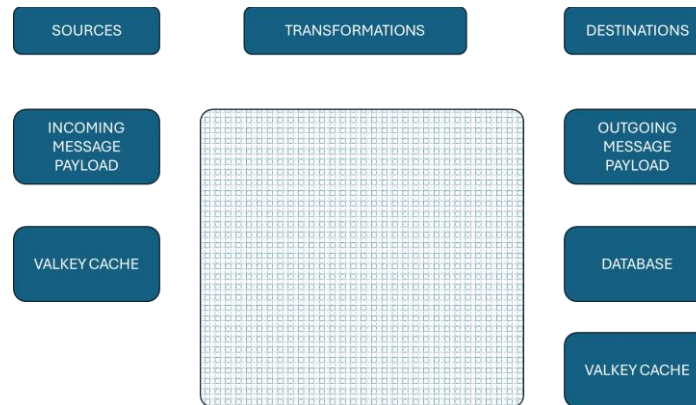
Source File Loading and Display

- The user begins by uploading a **sample input message** in JSON format (or XML if applicable).
- The system parses the payload and automatically renders the list of **inferred source fields** on the interface.
- Field paths are generated and displayed using **JSONPath notation** (e.g., \$.Transaction.Amount.Value) or **XPath** for XML.

- Users can expand/collapse hierarchical fields and select nested elements or entire branches for mapping.

Destination Format

- The Mapping Utility shall map to the predefined internal Tazama templates, including the existing Tazama data model along with the updates to the model that come with integrations. Upon selecting the destination schema, the interface shall allow the **selection and rendering of multiple destination structures** for field-level mapping. A single incoming message (e.g., pacs.008) may be mapped to multiple destination collections—some of which may be used more than once (e.g., debtor and creditor both mapping to the entity collection). Users shall be able to dynamically add new destinations as needed for comprehensive message decomposition.



Mapping Creation Experience

- Users can create mappings:
 - By selecting field(s) from the source panel and selecting destination field(s) to map it with.
 - Users may use a **dropdown menu** to specify any function they may want to apply between the fields (merge or split).
- For each mapping, a tooltip shows data type compatibility, transformation options (e.g., concatenating), and mapping status (valid/invalid).



Validation:

- Type mismatches will be flagged during the mapping creation phase.

One-to-One, One-to-Many, Many-to-One Mapping

Functional Behavior:

- **One-to-One:** Direct value transfer (e.g., source.amount → destination.transactionAmount).
- **One-to-Many:** One field split into multiple target fields across multiple destinations (e.g., full name into first and last names of debtor and creditor both mapping to the entity collection).
- **Many-to-One:** Multiple fields merged into one (e.g., customer ID and ID type → customer key).

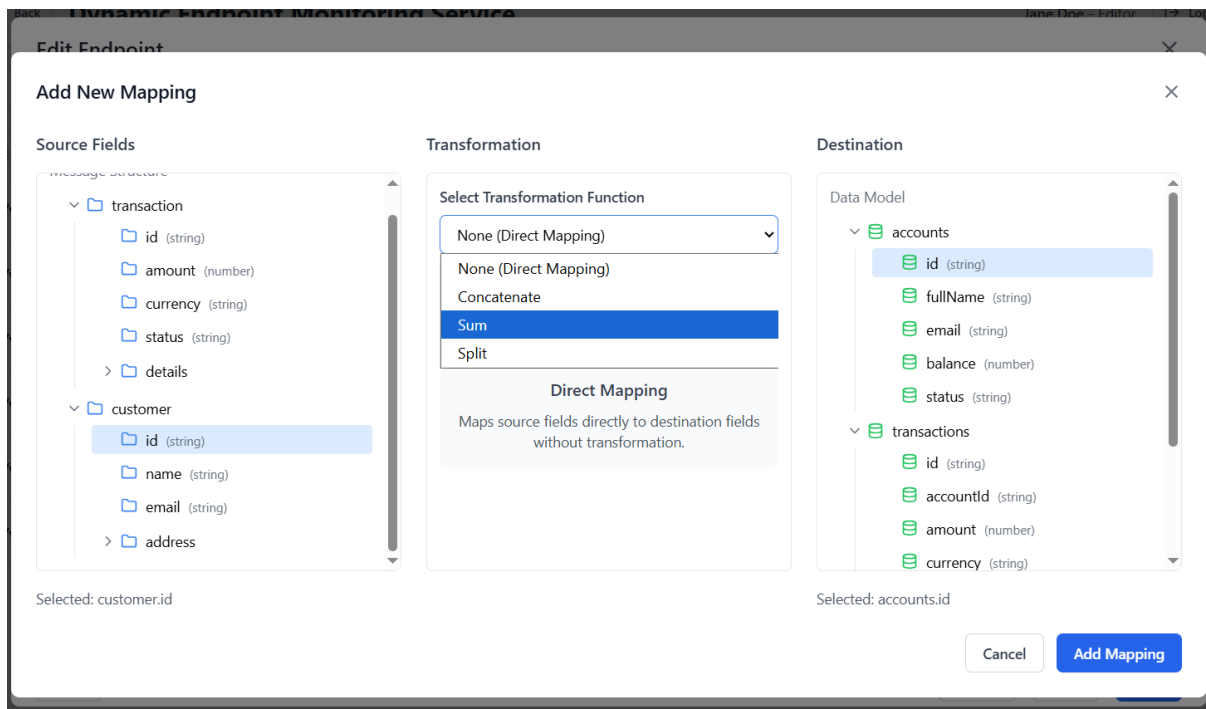


Figure 24. API Endpoint one-to-one field mapping

Technical Design:

- Transformation logic defined via a visual interface or mapping script DSL (Domain-Specific Language).
- Support for string concatenation, substring extraction, date formatting (format: YYYY-MM-DDT00:00:00.000Z), conditional value composition.
- Predefined transformation functions should include:
 - concat(), split(), etc.

Edge Case Handling:

- If one optional source field is missing in a many-to-one relationship, fallback or null-value handling logic must be configurable.

- If one mandatory source field/ destination field is missing in a many- to- one relationship, it should result in message rejection. Therefore, a mechanism for flagging fields as mandatory must be supported in the schema or configuration.

Constant Value Injection

Functional Behavior:

- Enables users to assign static/default values to destination fields that are:
 - Not present in the source payload.
 - Mandated by Tazama templates.

Technical Design:

- Constants can be string, numeric, boolean, or date types.
- Optional usage of variables (e.g., @currentTimestamp) or system-level tokens (e.g., @sourceDFSP) and Tenant ID.

Use Cases:

- Injecting default currency codes, system version tags, hardcoded message origins, etc.

Use of Tazama Message Templates

Functional Behavior:

- Mapping destination is defined based on standardized message templates available within Tazama:
 - e.g., Pain.001, Pain.013, Pacs.008, Pacs.002, etc.

Technical Design:

- Templates are stored as JSON Schemas or XSDs and loaded at runtime.
- The TCS allows switching between templates to reuse mappings or create variants.

Template Management:

- Templates can be versioned and extended via TCS configuration.

Dynamic Fields:

- Some fields are computed at runtime, others may be optional depending on use case profile (e.g., cross-border, high-value, etc.).

Output Validation

Functional Behavior:

- After mapping, the generated output is validated against the selected Tazama message template.

Validation Logic Includes:

- Schema compliance (field presence, data types, structure).
- Format constraints (e.g., ISO date, regex patterns, enumerations).
- Conditional constraints (e.g., if remittanceInfo exists, amountType must also be present).

Technical Implementation:

- Validation is executed pre-deployment and optionally during simulation (see 4.4.6).
- TCS must include built-in validators compliant with JSON Schema Draft 7 (or higher) and custom business rule validators.

Simulation/Dry-Run Mode

Functional Behavior:

- Users can execute a “dry-run” of the transformation pipeline with test payloads.
- Output is generated, validated, and displayed/logged without persisting or deploying the result.

Use Cases:

- Verifying mapping logic before publishing to production.
- Testing different variants of source data to evaluate mapping behavior.

The screenshot shows the 'Edit Endpoint' dialog in the 'Dynamic Endpoint Monitoring Service' interface. The dialog has a progress bar with four steps: 1. Payload, 2. Mapping, 3. Simulation (active), and 4. Deploy. The 'Test Mapping' section is visible, showing a 'Test Payload' field with a JSON object:

```
{  "transaction": {    "id": "TX12345",    "amount": 100.5,    "currency": "USD",    "status": "pending"  },  "customer": {    "id": "C789"  }}
```

. Below the payload field is a 'Run Simulation' button. A green message box indicates 'Validation Passed' with the text 'All fields have been successfully mapped and validated.' Below this is a 'Transformed Output' field showing the transformed JSON:

```
{  "id": "123456",  "value": 1000,  "currencyCode": "USD",  "channel": "MOBILE_APP"}
```

. At the bottom of the dialog are 'Back', 'Cancel', 'Save', and 'Next' buttons. The top of the dialog shows 'Jane Doe - Editor' and a 'Logout' button.

Features:

- Output preview with highlighting of mapped values and transformed structures.
- Side-by-side comparison of source vs. transformed message.
- Display of validation errors/warnings if applicable.

Versioning and Configuration Management

Functional Behavior:

- All mapping configurations are stored as version-controlled artifacts.

Capabilities:

- Version tracking: Users can view, tag, compare, and rollback mapping versions.
- Change logs and metadata (author, timestamp, description) must be recorded.
- Mappings tied to specific Tazama template versions to ensure forward/backward compatibility.

Importability and Exportability:

- Stored in a dedicated configuration store within the TCS host environment.
- Indexed by mapping ID, template name, version, creation date, and status (active/inactive).

Deployment Lifecycle:

- Mappings must be explicitly activated and bound to API endpoints after validation and approval.
- Support for staging environments prior to production deployment.

User-specific Actions

System Integrators

- Granted **full access** to the mapping utility, including:
 - One-to-one, one-to-many, many-to-one field relationships
 - Transformation logic using merging, splitting, and conditional rules
 - Constant injection, null-handling strategies, and use of external transformation functions
- Option to save mappings for reuse, clone, rollback, and apply to multiple templates.
- Have access to **all message templates** (standard and custom) for simulation.
- Can upload sample payloads and view both transformation output and validation logs.

4.5 Data Destination and Storage

The Tazama Connection Studio (TCS) configures the final storage of all transformed, validated, and enriched data within the Tazama ecosystem. The designated storage mechanism is the **Postgres database**, forming the Operational Data Store (ODS) for downstream consumption. This section outlines how and where data is stored, including rules, restrictions, and metadata handling.

Writing to Postgres Collections

Functional Behavior:

- All validated and mapped payloads processed through the TCS are written to specific **collections** within Postgres (best practices for writing to Postgres shall be followed).
- These collections act as the authoritative source of ingested data using Tazama services such as fraud detection, transaction scoring, and audit trail generation.

Technical Implementation:

- The TCS must connect to Postgres via a configured host URL, port, credentials, and target database name.
- Collections are defined by mapping configuration and linked to the endpoint or message type.
- Data is stored in document format (JSON), preserving the hierarchical structure of the mapped message.

Operational Features:

- Write operations must use **bulk inserts** for high-throughput scenarios in the enrichment layer.
- Indexed fields (e.g., transactionId, timestamp, dfspld) must be configured for each collection to enable efficient querying.
- Each stored document must include metadata such as:
 - ingestedAt (timestamp)
 - sourceSystem
 - MessageTemplateVersion

Restriction on External Writes

Functional Behavior:

- The TCS is explicitly restricted from writing data to any external runtime systems or databases outside of Postgres

Enforced Limitations:

- No integration or direct communication with:
 - Payment processors
 - Transaction management systems (at DEMS runtime)
 - External data lakes or third-party APIs

Dynamic Collection Creation with Schema Mapping

Functional Behavior:

- The TCS dynamically creates indexed Postgres collections if they don't exist at the time of endpoint deployment or message ingestion, for storing the message as received, and storing the updated transformed version of it as well.

Collection Naming Convention:

- Collection names follow a strict pattern:

 <tenantId>_<msgName>_<version>

e.g., tenant01_ingest_pain.001.01.011_v2

Schema Enforcement:

- The TCS uses the defined mapping schema to:
 - Infer the expected structure of the document.
 - Configure indexed fields for optimized access.
 - Apply validation rules during creation (e.g., presence of mandatory fields).

Collision Handling:

- If a collection with the same name already exists, warn user and abort creation unless override is explicitly allowed.
- Anyfield deletions from existing collections is prohibited without admin approval.

Internal Management:

- Created collections are registered in a metadata registry for traceability and lifecycle management.
- Each connection should store its required collections within its configuration to ensure independence and idempotency in database schema creation—i.e., it should be able to create its own database elements if they don't already exist. Since multiple connections may rely on the same database elements, the system must eventually consolidate all connection requirements into a unified and complete schema that supports all active connections.

Metadata Storage and Management

Functional Behavior:

- All configurations, mappings, endpoint and database definitions are treated as TCS artifacts and stored for traceability and version control.
- Delete and update operations on configuration artifacts shall be restricted; changes must be handled through version-controlled updates to ensure auditability and traceability.

Stored Metadata Includes:

- **Mapping Configurations:**
 - Source-to-destination mappings
 - Constant injections
 - Transformation rules
- **Endpoint Definitions:**
 - API path
 - Associated permissions for roles (RBAC)

- JSON schema
- Message template linkage
- **Enrichment Source Configuration:**
 - File path or URL
 - Ingestion schedule
 - Column-to-field mappings

Technical Implementation:

- Metadata is stored in **embedded databases** within the TCS host environment (Postgres).
- File format: JSON or YAML, organized by entity type and timestamp/version.
- Each artifact includes metadata tags:
 - createdBy, createdOn, lastModified, version, status

Security & Backup:

- Configuration files should be:
 - Backed up immediately.
 - Encrypted or access-restricted if sensitive data is present.
- TCS provides API commands for:
 - Listing artifacts
 - Exporting configurations
 - Importing/redeploying artifacts to another instance

Audit Trail:

- All changes to mappings or endpoints must be audit-logged and system-logged.
- Optional integration with a centralized logging system (e.g., Graylog, ELK) can be configured for governance.

5. Deployment Environments

Deployment of configurations, mappings, and API endpoints through the Tazama Connection Studio (TCS) follows a strict, role-based, and environment-aware workflow to ensure compliance with the Tazama multi-environment architecture.

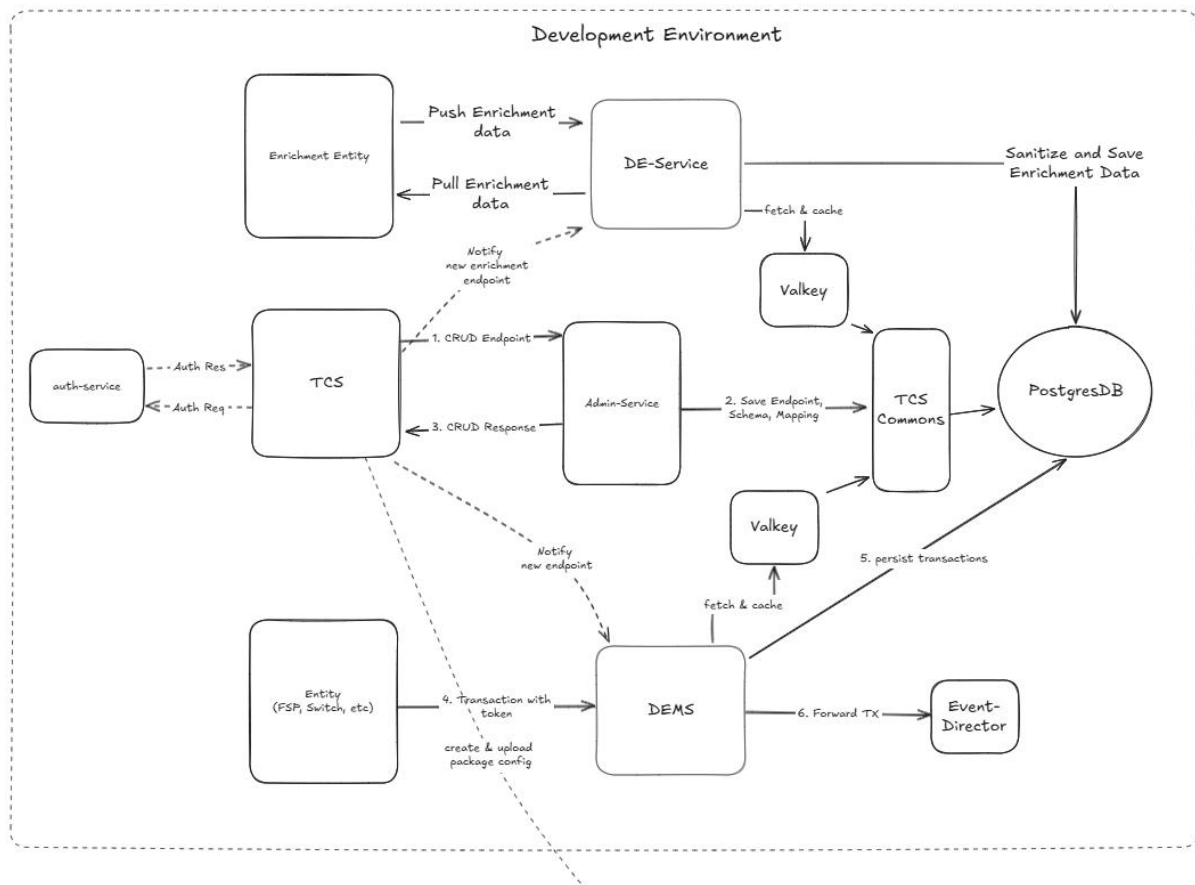
Deployment Across Environments

Deployments across environments (e.g., from Development to Production) are executed using configuration packages transferred via a secured SFTP server. In the Development Environment, the Exporter defines endpoint configurations, schemas, and mappings, which are stored in Postgres through the Admin-Service. Once finalized, these configurations are exported as a versioned configuration package and uploaded to the SFTP server.

In the Production Environment, the Importer retrieves the package from the SFTP, verifies its integrity, and pushes it through the Admin-Service to be sanitized and saved into the Production Postgres instance, ensuring controlled, auditable, and environment-agnostic deployment of runtime configurations.

Users Can Deploy Across Environments:

- **Editor** – Prepares and sends the configuration package to the Approver in the source environment.
- **Approver** – Validates and approves the configuration package, and sends it to the SFTP server for deployment.
- **Publisher** – A separate authorized role in the target environment (e.g., Production) responsible for publishing the received package.



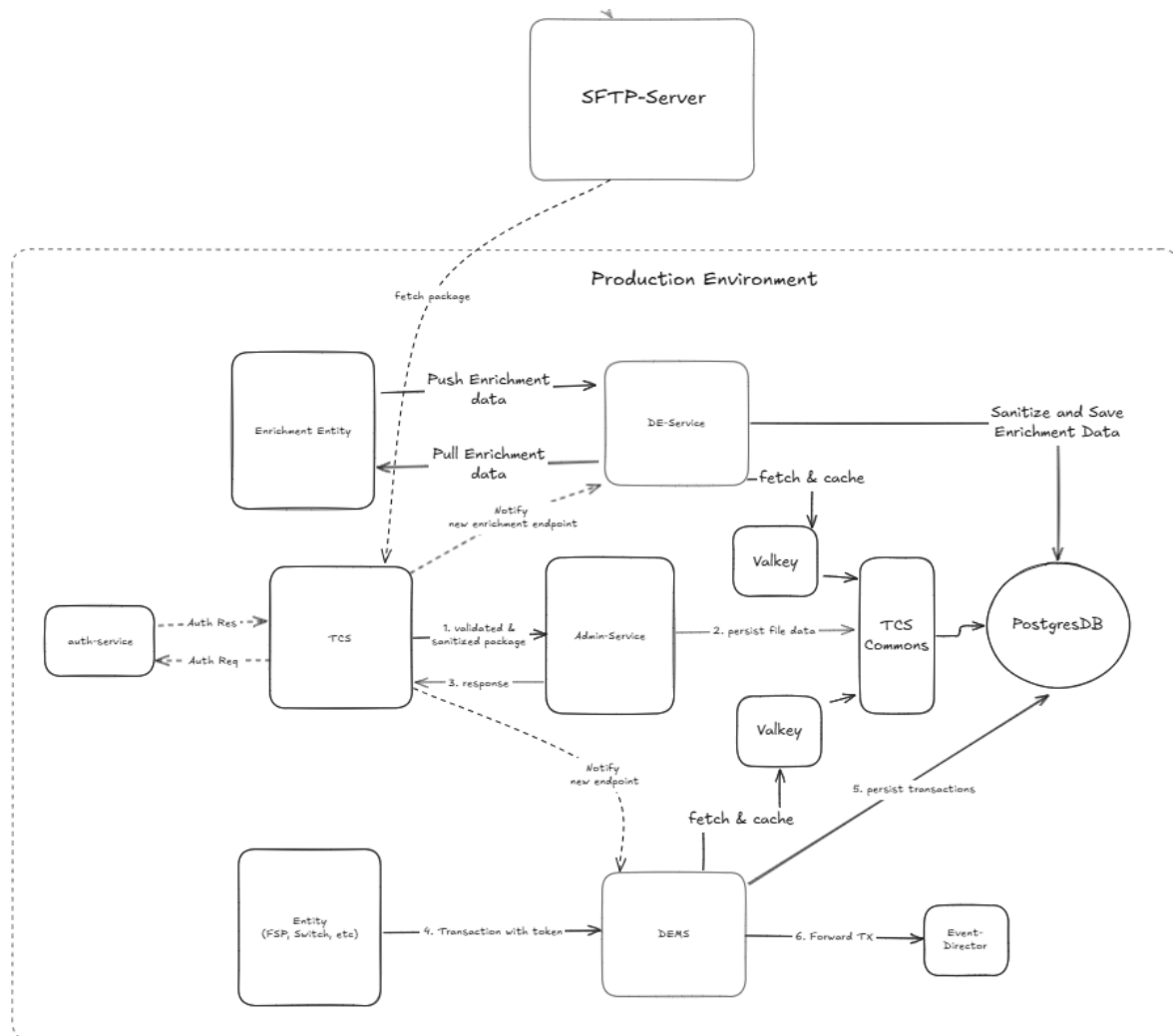


Figure 26. Deployment via SFTP

Workflow:

1. The **Editor** finalizes and sends the configuration package from the source environment (e.g., UAT).
 - a. The export package may include schema, field mappings, enrichment settings, metadata, templates and a readme file.
 - b. The package is signed and version-tagged for integrity and traceability.
2. The **Approver** validates the config package, approves it, and sends it to the SFTP server for deployment.
3. The exported file is securely transferred to the target environment (via SFTP).
4. The **Publisher** opens the TCS and **imports** the configuration package and **publishes** the configuration to the production instance.
 - a. The endpoint becomes live within the production DEMS API Layer.
 - b. All associated schemas and mappings are loaded into the designated production Postgres collections.
 - c. A cache of that endpoint is also maintained in DEMS Valkey cache.
 - d.
5. Once deployed, the new or updated endpoint is cached by the DEMS API and becomes available for transaction ingestion and routing in the target environment.

Caching:

The DEMS API shall cache configuration data (e.g., endpoint definitions, schemas, and mappings) in memory using **node-cache**. The cache is loaded on startup and may be refreshed manually or on a scheduled interval. Invalidation mechanisms shall be provided to ensure the cache remains consistent with the latest configurations stored in Postgres.

A single instance of **Valkey** shall be used for caching inter-message data and runtime context objects (e.g., session metadata, enrichment references) required during transaction processing.

Governance Enforcement:

- The users Exporter and Importer must not overlap within the same environment to ensure separation of duties.
- Each action—export, import, publish—is logged for audit purposes with full traceability across environments.

Additional Considerations

Implementation Plan

The deployment plan is open-ended, and two options are currently under consideration:

- **Centralized TCS across environments** – where a single, centrally managed deployment will be designed.
 - **Distributed TCS across environments** – where configuration sharing between environments may be facilitated via an SFTP-based approach.
- The specific implementation details will be finalized during the execution phase.

Package Format

Config packages are structured as versioned JSON/YAML artifacts. The format for the JSON package for Tazama would be;

```
{
  "meta": {
    "exportedAt": "2025-06-20T12:00:00Z",
    "exportedBy": "hasan",
    "version": "v1.0",
    "environment": "UAT"
  },
  "endpoint": {
    "path": "/transactions/pacs009",
    "method": "POST",
    "version": "1.0",
    "type": "MONITORING",
    "status": "DEVELOPMENT",
    "description": "Pacs.009 ingestion endpoint"
  },
  "message": {
    "name": "pacs009",
    "schema": { /* JSON schema here */ },
  }
}
```

```

        "type": "INITIATOR"
    },
    "messageRelation": {
        "credtrId": "Cdtr.Nm",
        "debtrId": "Dbtr.Nm",
        "credtrAccId": "CdtrAcct.Id.IBAN",
        "debtrAccId": "DbtrAcct.Id.IBAN",
        "TxTp": "CreditTransfer",
        "MsgId": "GrpHdr.MsgId",
        "CreDtTm": "GrpHdr.CreDtTm",
        "Amt": "IntrBkSttlmAmt.#text",
        "Ccy": "IntrBkSttlmAmt.@_Ccy",
        "PmtInfId": null,
        "EndToEndId": "CdtTrfTxInf.PmtId.EndToEndId",
        "TxSts": null
    },
    "cron": {
        "name": "pacs009 poll",
        "expression": "0 */5 * * *",
        "timezone": "UTC",
        "isActive": "ACTIVE"
    },
    "enrichment": {
        "type": "PULL",
        "schema": { /* enrichment schema JSON here */ }
    },
    "authMeta": {
        "type": "API_KEY",
        "credentials": {
            "apiKey": "YOUR_API_KEY"
        }
    }
}

```

Securing the Config Package

To ensure the integrity and authenticity of configuration packages being imported or exported across environments (Development, QA, Production), **cryptographic hashes** or **checksums** will be used, depending on the level of assurance required.

1. Hash Validation

- **Generation on Export**

When a configuration package is exported from an environment (e.g., Dev), a cryptographic hash (e.g., **SHA-256**) should be computed on the full contents of the package archive (e.g., .tar.gz or .zip). The generated hash value must be:

- Appended to a separate file (e.g., package.hash)
- Digitally signed (optional) if required for secure provenance

This ensures any post-export modification of the package can be detected before import.

- **Verification on Import**

Prior to importing a configuration package into a target environment:

- The system should recompute the hash of the received package

- Compare the computed value against the one provided in package.hash
If the values do not match, the import must be **rejected** and logged as a potential integrity violation.

The package hash file would be:
SHA256 (endpoint-config-export.json)= <computed hash>

2. Checksum Validation

- **Export:**
 - Generate a simple checksum (e.g., CRC32, Adler-32) of the config package.
 - Save it in a file (e.g., config-package.checksum).
- **Import:**
 - Recalculate the checksum on the received file.
 - Match it against the original value.
 - Reject if mismatched, indicating potential file corruption.

3. Optional Signing for Trust Validation

In environments where trust must be guaranteed between systems or teams, the exported hash file can be signed using a private key (e.g., PGP or JWT-based signing). The importing environment would then validate the signature using the corresponding public key to ensure the source is trusted.

4. Enforcement and Audit

- All package imports must undergo mandatory hash validation before processing.
- All validation results (pass/fail) must be logged, timestamped, and auditable.
- Failed attempts should trigger alerts to the deployment and security teams.

6. Assumptions and Dependencies

- All messages intended for ingestion and evaluation by the DEMS API shall be **pushed** into TCS generated REST API endpoints by upstream systems. The TCS shall not poll Kafka topics, message queues, or any system-specific transport layers.
- All deployment environments (e.g., Dev, UAT, Prod) into which the TCS deploys API endpoints or data pulling services must be pre-provisioned and exist before deployment.
- The TCS shall only support ingestion of structured, text-based files (e.g., CSV, TSV) where a column-to-field mapping can be defined.
- TCS shall allow XML ingestion only if it includes transformation into a DEMS-compatible JSON structure. It assumes XML messages are structured and mappable.
- For file-based ingestion (e.g., SFTP), required credentials, access permissions, and network availability must be provisioned externally.
- The TCS assumes that enrichment data is structurally clean and semantically valid for mapping and lookup.
- All transformations and message mappings rely on the availability of either a:
 - JSON schema
 - Existing template from a prior endpoint (e.g., Pain.001, Pacs.008)
- The TCS assumes these are provided and will not generate schemas from unstructured input.
- Any integration with payment schemes, core banking systems, or message switches shall be handled by separate components if needed (e.g., implementation-specific layers).

7. Glossary

- **Tazama Connection Studio (TCS):** A tool to design, configure, and deploy data ingestion setups like APIs and mappings.
- **Connection:** A complete setup (API, schema, mapping) created in TCS and deployed for data intake.
- **API (Traditional):** A standard way for external systems to access internal services.
- **Configured API:** A TCS-built custom API used for data ingestion — more than just a standard API.
- **DEMS:** The engine that runs live APIs, validates data, and forwards it for processing.
- **Admin Service:** Stores all TCS-created setups in the database but doesn't run them.
- **Publisher:** The person who sends the approved setup to go live.
- **Editor:** The person who creates and configures the connection.
- **Approver:** The person who reviews and approves the setup before it goes live.
- **Connection Lifecycle:** The steps a connection goes through — from creation to deployment.
- **Postgres:** The database where all setups, rules, and data are saved.
- **Configuration Runtime:** The setup phase — creating and preparing connections.
- **Production Runtime:** The live phase — when DEMS runs the deployed connections.
- **Configuration Package:** A bundle of all setup files (API, mappings, etc.) for deployment.
- **Dynamic Endpoint:** A flexible API created in TCS, not hardcoded — ready for different data flows.

- **Deployment Environment:** Where a connection runs — like Dev, UAT, or Production.
- **Valkey Cache:** A fast memory-based store used during live data processing.
- **Enrichment Layer (DEAPI):** A layer for uploading supporting data (e.g., KYC) — not for transactions.
- **Pull Job:** A scheduled task that fetches data from other systems.
- **Push Job:** A setup that waits for other systems to send data to it.
- **Simulation Mode:** A way to test if your connection works before going live.
- **Mapping Utility:** A tool to link fields from incoming data to the internal format.
- **Schema:** A blueprint that defines what structure the incoming data should follow.

Link to Prototypes:

Editor View: <https://project-api-endpoint-manager-dashboard-370.magicpatterns.app/dashboard>

Approver View: <https://project-api-endpoint-manager-dashboard-creator-view-609.magicpatterns.app/login>

Publisher View: <https://project-api-endpoint-manager-dashboard-approver-view-937.magicpatterns.app/login>

